

Exame Final

os 70 exercícios, resolvidos e explicados

A pool completa do exame final da Piscine, do nível 0 ao 5. Para cada exercício: o que o enunciado pede, como se pensa o problema, o código, e **a explicação de cada linha**.

Nível 0 11

Nível 1 12

Nível 2 15

Nível 3 15

Nível 4 11

Nível 5 6

Todo o código foi compilado com `cc -Wall -Wextra -Werror` sem um único aviso e testado contra os exemplos dos enunciados.

Índice

70 exercícios, por nível. Clica num nome para ir ao exercício, ou **passa o rato por cima para ler o enunciado completo** sem sair daqui. No fim há um índice alfabético.

N0 Nível 0

11 exercícios

hello	Escrever <code>Hello World!</code> e um newline.
only_z	Escrever um <code>z</code> . Sem newline.
only_a	Escrever um <code>a</code> . Sem newline.
aff_z	Mostrar o primeiro <code>z</code> da string — que acaba por ser sempre <code>z</code> .
ft_countdown	Escrever <code>9876543210</code> e um newline.
ft_print_numbers	Função que escreve <code>0123456789</code> . Sem <code>main</code> e sem newline.
maff_alpha	Escrever o alfabeto com as letras pares em maiúscula.
maff_revalpha	O mesmo, mas do <code>z</code> para o <code>a</code> .
aff_first_param	Mostrar o primeiro argumento.
aff_last_param	Mostrar o último argumento.
aff_a	Mostrar o primeiro <code>a</code> da string, se existir.

N1 Nível 1

12 exercícios

ft_strlen	Função que devolve o comprimento de uma string.
ft_putstr	Função que escreve uma string no ecrã.
ft_strcpy	Função que copia uma string para dentro de outra.
ft_swap	Função que troca o conteúdo de dois inteiros.
rev_print	Mostrar a string ao contrário.
ulstr	Trocar maiúsculas por minúsculas e vice-versa.
rotone	Cada letra passa à seguinte do alfabeto.
rot_13	Cada letra avança 13 posições, dando a volta.
repeat_alpha	Cada letra repete-se tantas vezes quanto a sua posição no alfabeto.
search_and_replace	Substituir uma letra por outra dentro de uma string.
first_word	Mostrar a primeira palavra da string.
fizzbuzz	Os números de 1 a 100, com <code>fizz</code> , <code>buzz</code> e <code>fizzbuzz</code> .

alpha_mirror	Cada letra troca pela simétrica do alfabeto.
ft_strrev	Função que inverte uma string no próprio sítio.
ft_strcmp	Função que compara duas strings.
max	Função que devolve o maior inteiro de um array.
ft_strdup	Função que devolve uma cópia nova de uma string.
ft_atoi	Função que converte texto em número inteiro.
do_op	Fazer uma conta com dois números e um operador.
last_word	Mostrar a última palavra da string.
inter	Os caracteres que aparecem nas duas strings, sem repetidos.
union	Os caracteres que aparecem numa string ou na outra, sem repetidos.
wdmatch	Ver se a primeira string se consegue escrever com os caracteres da segunda, pela ordem.
is_power_of_2	Função que diz se um número é potência de 2.
print_bits	Função que escreve um byte em binário.
reverse_bits	Função que inverte a ordem dos bits de um byte.
swap_bits	Função que troca as duas metades de um byte.

paramsum	Mostrar quantos argumentos o programa recebeu.
print_hex	Converter um número decimal para hexadecimal.
ft_atoi_base	Função que converte texto de qualquer base até 16 num inteiro.
tab_mult	Escrever a tabuada de um número.
pgcd	O maior divisor comum de dois números.
lcm	Função que devolve o mínimo múltiplo comum.
add_prime_sum	Somar todos os primos até ao número dado.
epur_str	A string com exactamente um espaço entre palavras.
expand_str	O mesmo, mas com três espaços entre palavras.
str_capitalizer	A primeira letra de cada palavra em maiúscula, o resto em minúscula.
rstr_capitalizer	A última letra de cada palavra em maiúscula, o resto em minúscula.
hidenp	Dizer se a primeira string está escondida na segunda.
ft_range	Função que cria um array com todos os inteiros entre dois valores.
ft_rrange	O mesmo array, pela ordem inversa.
ft_list_size	Função que conta os elementos de uma lista ligada.

N4 **Nível 4**

11 exercícios

sort_int_tab	Função que ordena um array de inteiros no próprio sítio.
fprime	Decompor um número nos seus factores primos.
rostring	Passar a primeira palavra para o fim da frase.
rev_wstr	Escrever as palavras pela ordem inversa.
ft_split	Função que parte uma string num array de palavras.
ft_itoa	Função que converte um inteiro numa string nova.
ft_list_foreach	Função que aplica uma função a cada elemento de uma lista.
ft_list_remove_if	Função que apaga da lista os elementos iguais a um dado de referência.
sort_list	Função que ordena uma lista ligada com um comparador dado.
flood_fill	Função que pinta uma zona ligada de um mapa a duas dimensões.
check_mate	Dizer se o rei está em xeque num tabuleiro dado.

N5 **Nível 5**

6 exercícios

brackets	Verificar se os parênteses de cada argumento estão bem fechados.
ft_itoa_base	Função que converte um inteiro numa string, em qualquer base de 2 a 16.
print_memory	Função que mostra um bloco de memória em hexadecimal e em texto.
options	Juntar opções da linha de comando num inteiro e mostrá-lo em binário.
brainfuck	Interpretador da linguagem Brainfuck.
rpn_calc	Avaliar uma expressão em notação polaca inversa.

1 Como o exame funciona

As cinco regras do mecanismo que mudam a forma de estudar.

1. **O exame sorteia um exercício de cada vez, de uma lista fechada.** Essa lista é a deste documento: 70 exercícios repartidos por seis níveis. Num exame vês entre seis e dezasseis deles.
2. **Começa no nível 0 e sobe um degrau de cada vez.** Só se desbloqueia o nível seguinte depois de validar o actual, e **não se pode saltar**. Um buraco no nível 1 custa mais do que não saber nada do nível 4.
3. **Todos os degraus valem o mesmo.** O `hello` dá tantos pontos como o `sort_list`. Há duas variantes do sistema de pontos — numa, falhar faz o exercício seguinte valer menos; noutra, as tentativas são ilimitadas ao mesmo valor. Em qualquer delas a conclusão é a mesma: **os níveis baixos decidem a nota**, porque são os primeiros e valem igual.
4. **A norminette não é corrigida.** O corretor compila com `cc -Wall -Wextra -Werror`, verifica que não foram usadas funções proibidas, e compara o output byte a byte com o dele. Mais nada. Podes usar `for`, declarar variáveis a meio da função e escrever funções longas. Escreve o que for mais rápido de escrever *certo*.
5. **Um aviso do compilador é um exercício falhado.** O `-Werror` transforma qualquer aviso em erro. O clássico é receber um parâmetro e não o usar — resolve-se com `(void)argv;` na primeira linha.

As duas linhas do enunciado que decidem tudo

`Expected files` diz exactamente que ficheiros entregas. Se disser `rot_13.c`, o ficheiro chama-se `rot_13.c` — com o underscore e sem mais nada na pasta.

`Allowed functions` diz o que podes usar. Se disser apenas `write`, um `printf` escondido num `#include` dá zero. Se disser `printf, atoi`, usa-os — poupas metade do trabalho.

O ritual, antes de submeter

1. `cc -Wall -Wextra -Werror ficheiro.c -o teste` — zero avisos.
2. Correr **todos** os exemplos do enunciado, sempre com `| cat -e`. O `$` mostra onde a linha acaba de verdade, e a newline a mais ou a menos é o erro número um deste exame.
3. Correr **sem argumentos** e com **argumentos a mais**. É o que ninguém testa.
4. Se o exercício for uma função, apagar o `main` de teste: o corretor tem o dele e dois `main` não linkam.
5. `ls` na pasta — só os ficheiros do `Expected files`.

2 As bases

Seis coisas. Com elas montam-se todos os exercícios até ao nível 3, e quase todos os outros.

1 · Escrever um caractere

Quase todos os exercícios só permitem `write`. O `write` é incómodo de usar directamente, por isso a primeira coisa que se escreve é sempre esta:

```
#include <unistd.h>

void    ft_putchar(char c)
{
    write(1, &c, 1);
}
```

`write(1, endereço, quantos)`: o `1` é o ecrã, o `&c` é "o sítio na memória onde o `c` está", e o último `1` é "escreve um byte a partir daí".

Para texto fixo não é preciso o helper — `write(1, "ola\n", 4)` escreve os quatro caracteres de uma vez. O `\n` conta como um.

2 · Uma string é uma fila de caracteres que acaba em zero

```
char *s = "abc";
/*  s[0]='a'  s[1]='b'  s[2]='c'  s[3]='\0'  */
```

O `'\0'` é o caractere de valor zero, e em C o zero significa "falso". É por isso que este ciclo percorre a string toda e pára sozinho no fim:

```
i = 0;
while (s[i])
{
    ft_putchar(s[i]);
    i++;
}
```

Este é o ciclo mais repetido do exame. Uma boa parte dos exercícios é este ciclo com uma ou duas linhas diferentes lá dentro.

3 · As letras são números seguidos

No computador, `'a'` é 97, `'b'` é 98, e assim por diante até `'z'` que é 122. As maiúsculas são as mesmas menos 32: `'A'` é 65. Os algarismos também são seguidos: `'0'` é 48. Isto permite fazer *aritmética* com letras, e resolve metade dos níveis 1 e 2.

Para...	Escreve-se	Porquê
saber se <code>c</code> é minúscula	<code>c >= 'a' && c <= 'z'</code>	está entre 97 e 122
saber se <code>c</code> é algarismo	<code>c >= '0' && c <= '9'</code>	está entre 48 e 57
minúscula → maiúscula	<code>c - 32</code>	$97 - 32 = 65$
maiúscula → minúscula	<code>c + 32</code>	
posição da letra no alfabeto	<code>c - 'a'</code>	$98 - 97 = 1$, logo <code>'b'</code> é a n.º 1
a letra n.º <code>i</code> do alfabeto	<code>'a' + i</code>	o inverso da linha de cima
algarismo <code>'7'</code> → número 7	<code>c - '0'</code>	$55 - 48 = 7$
número 7 → algarismo <code>'7'</code>	<code>n + '0'</code>	

Dar a volta ao alfabeto

Andar k letras à frente a partir do `'x'` sai fora do alfabeto. A receita que resolve isso sempre, sem um único `if`:

```
c = 'a' + (c - 'a' + k) % 26;
```

Lê-se: põe a letra na escala 0–25, anda k , dá a volta com o resto de 26, volta a pôr na escala das letras. Com `k = 1` é o `rotone`; com `k = 13` é o `rot_13`.

4 • Os argumentos da linha de comando

```
int main(int argc, char **argv)
```

`argv` é a lista de palavras escritas no terminal e `argc` é quantas são — **contando o nome do programa**:

```
$> ./ulstr "Ola Mundo"
argv[0] = "./ulstr"    argc = 2
argv[1] = "Ola Mundo"
```

É por isso que "se o número de argumentos não for 1" se escreve `if (argc != 2)`. Contar mal o `argc` custa o exercício inteiro e é dos erros mais frequentes.

5 • O esqueleto que serve para quase tudo

Quase todos os programas dos níveis 0 a 3 têm esta forma. Vale a pena sabê-la de cor: depois é só preencher o meio.

```

#include <unistd.h>

void    ft_putchar(char c)
{
    write(1, &c, 1);
}

int main(int argc, char **argv)
{
    int i;

    if (argc == 2)
    {
        i = 0;
        while (argv[1][i])
        {
            /* o exercicio e so isto */
            i++;
        }
        ft_putchar('\n');
        return (0);
    }
}

```

Repara onde está o `ft_putchar('\n')`: **fora** do `if`. É de propósito. Quase todos os enunciados dizem "se o número de argumentos não for 1, mostra só um newline" — pondo-o fora, os dois casos ficam resolvidos de uma vez.

6 · Escrever um número

Na maior parte dos exercícios não há `printf`. Esta função resolve-o, e é recursiva: chama-se a si própria.

```

void    ft_putnbr(int n)
{
    if (n >= 10)
        ft_putnbr(n / 10);
    ft_putchar(n % 10 + '0');
}

```

Com `n = 42`: como 42 é maior ou igual a 10, a função chama-se com `42 / 10 = 4`. Essa, como 4 é menor que 10, escreve logo o `'4'`. Depois regressa e escreve `42 % 10 = 2`. Sai `42`. A recursão existe aqui porque os algarismos saem ao contrário se se fizer à bruta, e assim arrumam-se sozinhos.

Se o número puder ser negativo

```
void ft_putnbr(int n)
{
    if (n == -2147483648)
    {
        write(1, "-2147483648", 11);
        return ;
    }
    if (n < 0)
    {
        ft_putchar('-');
        n = -n;
    }
    if (n >= 10)
        ft_putnbr(n / 10);
    ft_putchar(n % 10 + '0');
}
```

O caso do `-2147483648` existe porque esse número não tem positivo: `-(-2147483648)` dá `-2147483648` outra vez e o programa entra em ciclo infinito.

Duas ideias que aparecem a partir do nível 2

Pedir memória com `malloc`

Quando é preciso devolver algo que não cabe num `int` — uma string nova, um array — não serve declarar uma variável normal dentro da função: ela desaparece quando a função acaba. Pede-se memória com `malloc`, que devolve um pedaço que fica. O padrão tem sempre três passos, por esta ordem:

1. **contar** quantos elementos são precisos (numa string, **comprimento + 1**, para caber o `'\0'`);
2. **pedir** com `malloc(sizeof(tipo) * quantos)` e verificar se falhou;
3. **preencher** e devolver.

Esquecer o `+ 1` do `'\0'` é o erro mais comum do nível 2, e o pior é que o programa muitas vezes ainda parece funcionar.

Listas ligadas

Uma lista ligada é uma corrente: cada elo guarda um dado e o **endereço do elo seguinte**. O último aponta para `NULL`, que é o sinal de fim — o equivalente ao `'\0'` das strings. Por isso o ciclo de uma lista é sempre o mesmo:

```
while (lista)                /* enquanto nao for NULL */
{
    /* faz alguma coisa com lista->data */
    lista = lista->next;      /* passa ao elo seguinte */
}
```

N0 Nível 0

11 exercícios. Os mais simples da pool. Nenhum recebe texto para transformar: ou escrevem uma coisa fixa, ou leem um argumento.

N0 hello

Escrever `Hello World!` e um newline.

O QUE O ENUNCIADO PEDE

Escrever `Hello World!` seguido de um newline. Não recebe nada, não decide nada.

Funções permitidas: `write`.

```
$> ./hello | cat -e
Hello World!$
```

COMO PENSAR

- 1 O texto é sempre o mesmo, logo não é preciso ciclo nenhum: escreve-se de uma vez.
- 2 O `write` precisa de saber **quantos bytes** escrever. `Hello World!` são 12 caracteres, mais o `\n` são 13.
- 3 O `\n` conta como **um** caractere, não dois — o `\` e o `n` juntos são uma só coisa.

O CÓDIGO

hello.c

```
1 #include <unistd.h>
2
3 int main(void)
4 {
5     write(1, "Hello World!\n", 13);
6     return (0);
7 }
```

LINHA A LINHA

- 1 Dá acesso ao `write`. Sem esta linha o compilador não conhece a função e recusa.
- 3 Todo o programa começa no `main`. O `void` entre parênteses diz "não recebo argumentos".
- 5 Escreve 13 bytes a partir do início do texto. O `1` da frente é o ecrã.
- 6 Devolver `0` é a forma de dizer ao sistema "correu bem". Todos os `main` acabam assim.

Contar mal os bytes é a única maneira de falhar este exercício. Se puseres `12`, o newline não sai e o output fica errado; se puseres `14`, escreves um byte a mais.

Quem não quiser contar pode escrever caractere a caractere num ciclo — é mais lento de escrever, mas não se engana.

N0 only_z

Escrever um `z`. Sem newline.

O QUE O ENUNCIADO PEDE

Escrever um caractere `z` na saída padrão. O enunciado **não pede newline nenhum** — e é aí que está a armadilha.

Funções permitidas: `write`.

```
$> ./only_z ; echo "[fim]"
z[fim]
```

COMO PENSAR

- 1 Ler o enunciado até ao fim. Não diz "followed by a newline", ao contrário de quase todos os outros.
- 2 Um caractere, um `write`, um byte.

O CÓDIGO

only_z.c

```
1 #include <unistd.h>
2
3 int    main(void)
4 {
5     write(1, "z", 1);
6     return (0);
7 }
```

LINHA A LINHA

- 5 Um único byte. Repara que é `"z"` e não `"z\n"`.

Há quem falhe este por acrescentar o `\n` por hábito. **Antes de escrever a primeira linha, lê o enunciado inteiro** — este exercício existe para apanhar quem não o faz.

N0 only_a

Escrever um `a`. Sem newline.

O QUE O ENUNCIADO PEDE

Exactamente o mesmo que o `only_z`, com a letra `a`. Também sem newline.

O CÓDIGO

only_a.c

```
1 #include <unistd.h>
2
3 int    main(void)
4 {
5     write(1, "a", 1);
6     return (0);
7 }
```

LINHA A LINHA

- 5 Só muda a letra dentro das aspas.

N0 aff_z

Mostrar o primeiro `z` da string — que acaba por ser sempre `z`.

O QUE O ENUNCIADO PEDE

O enunciado diz três coisas: mostra o primeiro `z` que encontrares na string, seguido de newline; se não houver nenhum `z`, escreve `z` seguido de newline; se o número de parâmetros não for 1, escreve `z` seguido de newline.

Os três casos dão o mesmo resultado. Todos os exemplos do enunciado mostram `z$`. Este exercício é uma partida a quem começa a escrever antes de ler.

Funções permitidas: `write`.

```
$> ./aff_z "abc" | cat -e
z$
$> ./aff_z "zaz sent le poney" | cat -e
z$
$> ./aff_z | cat -e
z$
```

COMO PENSAR

- 1 Ler os três casos do enunciado e escrever o que sai em cada um.
- 2 Reparar que a resposta é a mesma nos três.
- 3 Escrever `z` e um newline. Fim.

O CÓDIGO

aff_z.c

```
1 #include <unistd.h>
2
3 int main(void)
4 {
5     write(1, "z\n", 2);
6     return (0);
7 }
```

LINHA A LINHA

- 3 Nem sequer é preciso receber o `argc` e o `argv`: o resultado não depende deles.
- 5 Dois bytes: o `z` e o newline.

Uma solução que procura mesmo o `z` na string também passa — só demora dez vezes mais a escrever. A lição serve para o exame inteiro: **lê o enunciado e todos os exemplos antes de codificar.**

N0 ft_countdown

Escrever `9876543210` e um newline.

O QUE O ENUNCIADO PEDE

Mostrar todos os algarismos por ordem decrescente, seguidos de um newline.

Funções permitidas: `write`.

```
$> ./ft_countdown | cat -e
9876543210$
```

COMO PENSAR

- 1 São dez caracteres seguidos, logo compensa um ciclo em vez de dez `write`.
- 2 A pergunta a fazer em qualquer ciclo é: **o que muda de uma volta para a outra?** Aqui é o caractere, que desce.
- 3 Os algarismos são números seguidos no computador (`'0'` é 48, `'9'` é 57), por isso `c--` passa de `'9'` para `'8'` sozinho.
- 4 O ciclo pára quando passar do `'0'`.

O CÓDIGO

ft_countdown.c

```
1 #include <unistd.h>
2
3 int main(void)
4 {
5     char c;
6
7     c = '9';
8     while (c >= '0')
9     {
10         write(1, &c, 1);
11         c--;
12     }
13     write(1, "\n", 1);
14     return (0);
15 }
```

LINHA A LINHA

- 5 `c` é declarado como `char` e não como `int` de propósito: assim o `write` da linha 10 pode escrevê-lo directamente.
- 7 Começa no caractere `'9'` — com plicas. `'9'` é o caractere, `9` seria o número nove, que é outra coisa.
- 8 Continua enquanto `c` ainda for maior ou igual a `'0'`. Quando `c` desce abaixo de `'0'`, pára.
- 10 `&c` é "o sítio onde o `c` está guardado". O `write` precisa do sítio, não do valor.
- 11 Desce uma posição: de `'9'` para `'8'`, e por aí fora.
- 13 O newline final, já fora do ciclo — sai uma vez só.

N0 ft_print_numbers

Função que escreve `0123456789`. Sem `main` e sem newline.

O QUE O ENUNCIADO PEDE

Escrever uma **função** que mostra todos os algarismos por ordem crescente. O enunciado dá a declaração: `void ft_print_numbers(void);`

Repara em duas coisas: é uma função, não um programa, e não pede newline.

COMO PENSAR

- ❶ É o `ft_countdown` ao contrário: começa no `'0'` e sobe até ao `'9'`.
- ❷ Como é uma função, **não se escreve** `main`. Quem chama é o corretor.
- ❸ Como o enunciado não pede newline, não se escreve newline.

O CÓDIGO

ft_print_numbers.c

```
1 #include <unistd.h>
2
3 void ft_print_numbers(void)
4 {
5     char c;
6
7     c = '0';
8     while (c <= '9')
9     {
10         write(1, &c, 1);
11         c++;
12     }
13 }
```

LINHA A LINHA

- 3 A declaração tem de ser exactamente a do enunciado: devolve `void` (nada) e não recebe nada.
-
- 7-8 Começa no `'0'` e sobe enquanto não passar do `'9'`.
-
- 11 `c++` sobe uma posição no alfabeto dos algarismos.

"Function" ou "program"? Olha sempre para a linha `Expected files` e para a declaração do enunciado. Se o enunciado te dá uma declaração, entregas só a função: o corretor tem o `main` dele. Se entregares um `main` também, ficam dois no mesmo programa, nem sequer linka, e é zero.

N0 maff_alpha

Escrever o alfabeto com as letras pares em maiúscula.

O QUE O ENUNCIADO PEDE

Mostrar o alfabeto com as letras de ordem par em maiúscula e as de ordem ímpar em minúscula, seguido de um newline.

Funções permitidas: `write`.

```
$> ./maff_alpha | cat -e
aBcDeFgHiJkLmNoPqRsTuVwXyZ$
```

COMO PENSAR

- 1 São 26 letras — um ciclo, não 26 `write`.
- 2 O truque está em **contar a posição no alfabeto em vez da letra**. Se `i` for 0, 1, 2..., então `'a' + i` dá a i-ésima minúscula e `'A' + i` dá a i-ésima maiúscula.
- 3 `i % 2` (o resto da divisão por 2) vale 0 nas posições pares e 1 nas ímpares. É isso que decide a caixa.
- 4 Cuidado com a contagem: o `a` é a **primeira** letra, logo é ímpar e fica minúscula. Com `i` a começar em 0, isso corresponde a `i % 2 == 0`.

O CÓDIGO

maff_alpha.c

```
1 #include <unistd.h>
2
3 void ft_putchar(char c)
4 {
5     write(1, &c, 1);
6 }
7
8 int main(void)
9 {
10     int i;
11
12     i = 0;
13     while (i < 26)
14     {
15         if (i % 2 == 0)
16             ft_putchar('a' + i);
17         else
18             ft_putchar('A' + i);
19         i++;
20     }
21     ft_putchar('\n');
22     return (0);
23 }
```

LINHA A LINHA

3-6 O helper de sempre. A partir daqui escreve-se `ft_putchar(x)` em vez de `write(1, &x, 1)`.

10 `i` não é a letra, é a **posição** dela no alfabeto: o para o `a`, 25 para o `z`.

13 26 voltas, uma por letra.

15 Resto da divisão por 2: se der 0, é uma posição par na contagem a começar em zero.

16 `'a' + i` dá a i-ésima minúscula. Com `i = 0` dá `'a'`, com `i = 2` dá `'c'`.

18 `'A' + i` dá a i-ésima maiúscula. Com `i = 1` dá `'B'`, com `i = 3` dá `'D'`.

21 O newline, uma vez só, no fim.

Sem a ideia de contar a posição em vez da letra, este exercício obrigava a 26 `if`. É o mesmo truque que resolve o `rot_13` e o `alpha_mirror` mais à frente.

N0 maff_revalpha

O mesmo, mas do `z` para o `a`.

O QUE O ENUNCIADO PEDE

Mostrar o alfabeto ao contrário, com as letras de ordem par em maiúscula, seguido de um newline.

```
$> ./maff_revalpha | cat -e
zYxWvUtSrQpOnMlKjIhGfEdCbA$
```

COMO PENSAR

- 1 É o `maff_alpha` com uma mudança só: em vez de somar a partir do princípio do alfabeto, subtrai-se a partir do fim.

O CÓDIGO

maff_revalpha.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  int     main(void)
9  {
10     int i;
11
12     i = 0;
13     while (i < 26)
14     {
15         if (i % 2 == 0)
16             ft_putchar('z' - i);
17         else
18             ft_putchar('Z' - i);
19         i++;
20     }
21     ft_putchar('\n');
22     return (0);
23 }
```

LINHA A LINHA

16 `'z' - i` conta para trás a partir do `z`: com `i = 0` dá `'z'`, com `i = 2` dá `'x'`.

18 `'Z' - i` faz o mesmo nas maiúsculas: com `i = 1` dá `'Y'`.

N0 aff_first_param

Mostrar o primeiro argumento.

O QUE O ENUNCIADO PEDE

Mostrar o primeiro argumento seguido de um newline. Se não houver argumentos, mostrar só o newline.

Funções permitidas: `write`.

```
$> ./aff_first_param vincent mit "l'ane" dans un pre | cat -e
vincent$
$> ./aff_first_param | cat -e
$
```

COMO PENSAR

- 1 `argv[0]` é o nome do programa, por isso o primeiro argumento a sério é `argv[1]`.
- 2 `argv[1]` é uma string, e uma string percorre-se com o ciclo `while (s[i])`.
- 3 O newline tem de sair nos dois casos. Pondo-o fora do `if`, resolvem-se os dois de uma vez.

O CÓDIGO

aff_first_param.c

```
1 #include <unistd.h>
2
3 int main(int argc, char **argv)
4 {
5     int i;
6
7     if (argc >= 2)
8     {
9         i = 0;
10        while (argv[1][i])
11        {
12            write(1, &argv[1][i], 1);
13            i++;
14        }
15    }
16    write(1, "\n", 1);
17    return (0);
18 }
```

LINHA A LINHA

3 `argc` é quantas palavras foram escritas no terminal, **incluindo o nome do programa**. `argv` é a lista dessas palavras.

7 "Há pelo menos um argumento" escreve-se `argc >= 2`, porque o `argv[0]` também conta.

10 Percorre `argv[1]` até ao `'\0'` que marca o fim da string.

12 `&argv[1][i]` é o sítio onde está o caractere número `i` do primeiro argumento.

16 **Fora do `if`**. É isto que faz o segundo exemplo funcionar sem escrever código extra.

N0 aff_last_param

Mostrar o último argumento.

O QUE O ENUNCIADO PEDE

O mesmo, mas o último argumento em vez do primeiro.

```
$> ./aff_last_param "zaz" "mange" "des" "chats" | cat -e
chats$
```

COMO PENSAR

- 1 Se `argc` é quantas palavras há ao todo, a última está em `argv[argc - 1]`.
- 2 O resto é igual ao `aff_first_param`.

O CÓDIGO

aff_last_param.c

```
1 #include <unistd.h>
2
3 int main(int argc, char **argv)
4 {
5     int i;
6
7     if (argc >= 2)
8     {
9         i = 0;
10        while (argv[argc - 1][i])
11        {
12            write(1, &argv[argc - 1][i], 1);
13            i++;
14        }
15    }
16    write(1, "\n", 1);
17    return (0);
18 }
```

LINHA A LINHA

- 10 `argc - 1` é o índice da última palavra. Com quatro argumentos, `argc` é 5 e a última está em `argv[4]`.
- 12 A mesma troca dentro do `write`.

N0 aff_a

Mostrar o primeiro `a` da string, se existir.

O QUE O ENUNCIADO PEDE

Mostrar o primeiro caractere `a` encontrado na string, seguido de newline. Se não houver nenhum `a`, mostrar apenas o newline. Se o número de parâmetros não for 1, mostrar `a` seguido de newline.

Ao contrário do `aff_z`, aqui os três casos dão resultados **diferentes**.

Funções permitidas: `write`.

```
$> ./aff_a "abc" | cat -e
a$
$> ./aff_a "zz sent le poney" | cat -e
$
$> ./aff_a | cat -e
a$
```

COMO PENSAR

- 1 Escrever os três casos numa folha antes de codificar. São fáceis de baralhar.
- 2 O caso "não é exactamente 1 argumento" resolve-se logo no início e o programa acaba ali.
- 3 Nos outros dois, percorre-se a string à procura do primeiro `a`. Se aparecer, escreve-se e **pára-se** — senão uma string com dois `a` escrevia dois.
- 4 O newline sai sempre no fim.

O CÓDIGO

aff_a.c

```
1 #include <unistd.h>
2
3 int main(int argc, char **argv)
4 {
5     int i;
6
7     if (argc != 2)
8     {
9         write(1, "a\n", 2);
10        return (0);
11    }
12    i = 0;
13    while (argv[1][i])
14    {
15        if (argv[1][i] == 'a')
16        {
17            write(1, "a", 1);
18            break ;
19        }
20        i++;
21    }
22    write(1, "\n", 1);
23    return (0);
24 }
```

LINHA A LINHA

- | | |
|-------|--|
| 7 | "Exactamente 1 argumento" é <code>argc == 2</code> . Aqui testa-se o contrário. |
| 9-10 | Escreve <code>a</code> e newline de uma vez e acaba o programa. O <code>return</code> faz com que nada do resto corra. |
| 13 | O ciclo de sempre a percorrer a string. |
| 15 | Compara o caractere actual com <code>'a'</code> — plicas, porque é um caractere e não uma string. |
| 17-18 | Escreve o <code>a</code> e sai do ciclo. O <code>break</code> é o que garante que só sai um. |
| 22 | O newline final, comum aos dois casos que chegam aqui. |

N1 Nível 1

12 exercícios. Aparece a string. A partir daqui quase tudo é percorrer caracteres, e começam as funções sem `main`.

N1 ft_strlen

Função que devolve o comprimento de uma string.

O QUE O ENUNCIADO PEDE

Escrever uma função que devolve o comprimento de uma string. O enunciado dá a declaração: `int`

`ft_strlen(char *str);`

Não é um programa: não há `main`, não há `write`, não há nada para mostrar no ecrã. Só se devolve um número.

COMO PENSAR

- 1 Uma string acaba sempre num caractere invisível de valor zero, o `'\0'`. O comprimento é quantos caracteres existem **antes** dele.
- 2 Logo: andar do início até encontrar o zero, contando os passos.
- 3 O contador começa em 0 e acaba exactamente no índice do `'\0'`, que por acaso é o comprimento. Não é preciso somar nem subtrair nada no fim.

O CÓDIGO

ft_strlen.c

```
1 int ft_strlen(char *str)
2 {
3     int i;
4
5     i = 0;
6     while (str[i])
7         i++;
8     return (i);
9 }
```

LINHA A LINHA

- 1 A declaração exacta do enunciado. `char *str` é "o endereço do primeiro caractere da string".
- 3 O contador. Vai servir de índice e de resposta ao mesmo tempo.
- 5 Começa no primeiro caractere, que é o índice 0. Em C conta-se sempre a partir do zero.
- 6 `while (str[i])` lê-se "enquanto o caractere número `i` não for o zero final". Em C, o valor 0 conta como falso, por isso o ciclo pára sozinho no `'\0'`.
- 7 Um passo à frente. Repara que não há chavetas: quando o corpo do `while` é uma linha só, elas são opcionais.
- 8 Quando o ciclo pára, `i` é o número de caracteres que foram contados. Devolve-se esse número.

Esta função aparece dentro de metade dos exercícios seguintes. Vale a pena sabê-la de cor.

N1 ft_putstr

Função que escreve uma string no ecrã.

O QUE O ENUNCIADO PEDE

Escrever uma função que mostra uma string na saída padrão. A declaração é `void ft_putstr(char *str);`

Funções permitidas: `write`.

COMO PENSAR

- 1 É o mesmo ciclo do `ft_strlen`, mas em vez de contar, escreve.
- 2 Como não devolve nada, o tipo é `void`.

O CÓDIGO

ft_putstr.c

```
1 #include <unistd.h>
2
3 void ft_putstr(char *str)
4 {
5     while (*str)
6     {
7         write(1, str, 1);
8         str++;
9     }
10 }
```

LINHA A LINHA

- 3 `void` quer dizer "não devolvo nada". Por isso não há `return` no fim.
- 5 `*str` é "o caractere que está no sítio para onde o `str` aponta". É outra maneira de escrever `str[0]`.
- 7 O `write` quer um **endereço**, e o `str` já é um endereço — por isso aqui não se põe o `&`.
- 8 `str++` faz o ponteiro avançar uma posição, e a partir daí `*str` passa a ser o caractere seguinte.

Podia escrever-se exactamente igual com um índice: `while (str[i])` e `write(1, &str[i], 1)`. As duas versões fazem o mesmo e as duas passam. Usa a que perceberes melhor sob pressão.

N1 ft_strcpy

Função que copia uma string para dentro de outra.

O QUE O ENUNCIADO PEDE

Reproduzir o comportamento do `strcpy` do `man`: copia a string `s2` para dentro de `s1` e devolve o `s1`.

A declaração é `char *ft_strcpy(char *s1, char *s2);`

COMO PENSAR

- 1 Percorrer o `s2` e ir copiando cada caractere para a mesma posição do `s1`.
- 2 O ciclo pára no `'\0'` do `s2`, o que significa que o `'\0'` **não é copiado**. Tem de se pôr à mão no fim.
- 3 Devolve-se o `s1`, tal como o enunciado manda.

O CÓDIGO

ft_strcpy.c

```
1 char *ft_strcpy(char *s1, char *s2)
2 {
3     int i;
4
5     i = 0;
6     while (s2[i])
7     {
8         s1[i] = s2[i];
9         i++;
10    }
11    s1[i] = '\0';
12    return (s1);
13 }
```

LINHA A LINHA

- 1 Devolve `char *`, ou seja, um endereço de string — o mesmo que recebeu em `s1`.
- 6 Anda enquanto houver caracteres no `s2`. É o `s2` que manda, porque é ele que está a ser copiado.
- 8 Copia um caractere. Os dois índices são o mesmo `i`, por isso a posição é a mesma nas duas strings.
- 11 **A linha que toda a gente esquece.** Sem ela o `s1` não tem fim marcado, e quem o for imprimir escreve lixo até encontrar um zero por acaso.
- 12 Devolve o ponteiro recebido, como o enunciado pede.

O `ft_strcpy` não pede memória: quem chama a função já tem de ter espaço suficiente no `s1`. Não confundir com o `ft_strdup`, no nível 2, que **cria** a string nova.

N1 ft_swap

Função que troca o conteúdo de dois inteiros.

O QUE O ENUNCIADO PEDE

Escrever uma função que troca o conteúdo de dois inteiros cujos endereços são passados como parâmetros.

A declaração é `void ft_swap(int *a, int *b);`

COMO PENSAR

- ❶ Os parâmetros são **endereços** (`int *`), não valores. `*a` significa "o valor que está nesse endereço".
- ❷ Se a função recebesse `int a, int b`, trocava as cópias dela e o programa de quem chamou não mudava nada. É por isso que o enunciado pede ponteiros.
- ❸ Trocar dois valores precisa sempre de um terceiro sítio para guardar um deles: se se fizer `*a = *b` primeiro, o valor de `a` perde-se.

O CÓDIGO

ft_swap.c

```
1 void ft_swap(int *a, int *b)
2 {
3     int tmp;
4
5     tmp = *a;
6     *a = *b;
7     *b = tmp;
8 }
```

LINHA A LINHA

- 1 `int *a` é "o endereço de um inteiro". O `*` faz parte do tipo.
- 3 A variável temporária. Sem ela a troca é impossível.
- 5 Guarda o valor de `a` de lado.
- 6 Põe o valor de `b` em `a`. Neste momento o valor original de `a` já só existe no `tmp`.
- 7 Põe o valor guardado em `b`. Troca feita.

Para testar, quem chama escreve `ft_swap(&x, &y)` — o `&` é o que transforma uma variável no endereço dela.

N1 rev_print

Mostrar a string ao contrário.

O QUE O ENUNCIADO PEDE

Mostrar a string recebida ao contrário, seguida de um newline. Se o número de parâmetros não for 1, mostrar só o newline.

Funções permitidas: `write`.

```
$> ./rev_print "dub0 a POIL" | cat -e
LIOP a 0bud$
$> ./rev_print | cat -e
$
```

COMO PENSAR

- 1 Para escrever ao contrário é preciso começar no fim, e para chegar ao fim é preciso primeiro medir a string.
- 2 Medir é o `ft_strlen` feito à mão: andar até ao `'\0'`.
- 3 Aí o índice está **em cima** do `'\0'`, que não se quer escrever. Por isso recua-se um.
- 4 Depois anda-se para trás até passar do índice 0.

O CÓDIGO

rev_print.c

```
1 #include <unistd.h>
2
3 int main(int argc, char **argv)
4 {
5     int i;
6
7     if (argc == 2)
8     {
9         i = 0;
10        while (argv[1][i])
11            i++;
12        i--;
13        while (i >= 0)
14        {
15            write(1, &argv[1][i], 1);
16            i--;
17        }
18    }
19    write(1, "\n", 1);
20    return (0);
21 }
```

LINHA A LINHA

7 "Exactamente 1 argumento" é `argc == 2`, porque o `argv[0]` é o nome do programa.

10-11 Mede a string: anda até ao `'\0'`. No fim, `i` é o comprimento.

12 **Recua um.** Sem esta linha o primeiro caractere a sair seria o `'\0'`, e o output ficava errado.

13 Anda para trás enquanto ainda estiver dentro da string. Quando `i` chega a `-1`, pára.

15 Escreve o caractere número `i`. O `&` dá o endereço dele, que é o que o `write` quer.

19 O newline **fora** do `if`: assim sai tanto no caso normal como no caso sem argumentos.

N1 **ulstr**

Trocar maiúsculas por minúsculas e vice-versa.

O QUE O ENUNCIADO PEDE

Mostrar a string com a caixa de todas as letras invertida: as minúsculas passam a maiúsculas e as maiúsculas a minúsculas. Tudo o que não for letra fica igual. No fim, um newline.

Se o número de argumentos não for 1, mostrar só o newline.

Funções permitidas: `write`.

```
$> ./ulstr "3:21 Ba tOut moUn ki Ka di KE" | cat -e
3:21 bA ToUT MOuN KI kA DI ke$
```

COMO PENSAR

- 1 Este é o padrão do exame: percorrer a string e, para cada caractere, decidir o que escrever. Vários exercícios a seguir são este mesmo ciclo com duas linhas diferentes lá dentro.
- 2 A decisão aqui: se é minúscula, sobe; se é maiúscula, desce; senão, fica igual.
- 3 As maiúsculas estão exactamente 32 abaixo das minúsculas na tabela de caracteres, por isso a conversão é uma soma ou uma subtracção.

O CÓDIGO

ulstr.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  int     main(int argc, char **argv)
9  {
10     int    i;
11     char   c;
12
13     if (argc == 2)
14     {
15         i = 0;
16         while (argv[1][i])
17         {
18             c = argv[1][i];
19             if (c >= 'a' && c <= 'z')
20                 c = c - 32;
21             else if (c >= 'A' && c <= 'Z')
22                 c = c + 32;
23             ft_putchar(c);
24             i++;
25         }
26     }
27     ft_putchar('\n');
28     return (0);
29 }
```

LINHA A LINHA

10-11 Duas variáveis: o índice que anda na string e uma cópia do caractere actual para poder ser modificado.

16 O ciclo de sempre: anda até ao `'\0'`.

18 Tira uma cópia do caractere. Mexe-se na cópia, não no argumento original.

19-20 Se está entre `'a'` e `'z'`, é minúscula, logo `- 32` sobe para maiúscula.

21-22 Senão, se está entre `'A'` e `'Z'`, é maiúscula, logo `+ 32` desce para minúscula.

23 Escreve o caractere, mudado ou não. Tudo o que não é letra chega aqui sem ter passado por nenhum dos dois `if`.

27 O newline final, fora do `if` do `argc`.

O `else if` da linha 21 é obrigatório. Com dois `if` separados, uma minúscula passava a maiúscula no primeiro e voltava a minúscula no segundo — o programa não fazia nada.

N1 rotone

Cada letra passa à seguinte do alfabeto.

O QUE O ENUNCIADO PEDE

Mostrar a string com cada letra substituída pela seguinte do alfabeto. O `z` passa a `a` e o `Z` passa a `A`. A caixa não muda e o que não for letra fica igual. No fim, um newline.

Se o número de argumentos não for 1, mostrar só o newline.

Funções permitidas: `write`.

```
$> ./rotone "AkjhZ zLKIJz , 23y " | cat -e
BlkiA aMLJKa , 23z $
```

COMO PENSAR

- 1 É o mesmo ciclo do `ulstr`, só muda a decisão lá dentro.
- 2 Somar 1 resolve quase tudo, **menos** no fim do alfabeto: `'z' + 1` daria `'{'`, que não é letra nenhuma.
- 3 Por isso os dois casos da ponta tratam-se primeiro, e só depois o caso geral.

O CÓDIGO

rotone.c

```
1 #include <unistd.h>
2
3 void ft_putchar(char c)
4 {
5     write(1, &c, 1);
6 }
7
8 int main(int argc, char **argv)
9 {
10     int i;
11     char c;
12
13     if (argc == 2)
14     {
15         i = 0;
16         while (argv[1][i])
17         {
18             c = argv[1][i];
19             if (c == 'z')
20                 c = 'a';
21             else if (c == 'Z')
22                 c = 'A';
23             else if ((c >= 'a' && c < 'z') || (c >= 'A' && c < 'Z'))
24                 c = c + 1;
25             ft_putchar(c);
26             i++;
27         }
28     }
29     ft_putchar('\n');
30     return (0);
31 }
```

LINHA A LINHA

- 19-20 O caso especial do fim do alfabeto em minúsculas: o `z` volta ao princípio.
- 21-22 O mesmo nas maiúsculas.
- 23 Só chega aqui quem não é `z` nem `Z`. O teste usa `<` e não `<=` de propósito: o `z` já foi tratado.
- 24 O caso geral: a letra seguinte é a de código imediatamente acima.

Há uma alternativa sem casos especiais nenhuns, com a fórmula do resto: `c = 'a' + (c - 'a' + 1) % 26;` para as minúsculas e o equivalente para as maiúsculas. É mais curta, e é obrigatória no `rot_13` — aí o salto é de 13 e os casos especiais seriam 13 de cada lado.

O QUE O ENUNCIADO PEDE

Mostrar a string com cada letra substituída pela letra 13 posições à frente no alfabeto. O `z` passa a `m` e o `Z` passa a `M`. A caixa não muda. No fim, um newline.

Se o número de argumentos não for 1, mostrar só o newline.

Funções permitidas: `write`.

```
$> ./rot_13 "My horse is Amazing." | cat -e
Zl ubefr vf Nznmvat.$
```

COMO PENSAR

- 1 É o `rotone` com 13 em vez de 1, mas agora os casos especiais seriam 13 letras de cada lado — impraticável.
- 2 A solução é a fórmula que dá a volta ao alfabeto sozinha. Lê-se em três tempos: põe a letra na escala 0 a 25, anda 13, e se passar do 25 dá a volta com o resto da divisão por 26. No fim volta-se a pôr na escala das letras.

O CÓDIGO`rot_13.c`

```
1 #include <unistd.h>
2
3 void ft_putchar(char c)
4 {
5     write(1, &c, 1);
6 }
7
8 int main(int argc, char **argv)
9 {
10     int i;
11     char c;
12
13     if (argc == 2)
14     {
15         i = 0;
16         while (argv[1][i])
17         {
18             c = argv[1][i];
19             if (c >= 'a' && c <= 'z')
20                 c = 'a' + (c - 'a' + 13) % 26;
21             else if (c >= 'A' && c <= 'Z')
22                 c = 'A' + (c - 'A' + 13) % 26;
23             ft_putchar(c);
24             i++;
25         }
26     }
27     ft_putchar('\n');
28     return (0);
29 }
```

LINHA A LINHA

- 20 `c - 'a'` põe a letra na escala 0 a 25. Soma-se 13. O `% 26` é o que dá a volta: 25 + 13 dá 38, e 38 a dividir por 26 deixa resto 12. Finalmente `'a' +` volta a pôr na escala das letras.
- 22 Exactamente o mesmo nas maiúsculas, mudando só o ponto de partida de `'a'` para `'A'`.

Esta fórmula serve para qualquer salto. Trocando o 13 por 1 obtém-se o `rotone`; a estrutura do exercício é a mesma.

N1 repeat_alpha

Cada letra repete-se tantas vezes quanto a sua posição no alfabeto.

O QUE O ENUNCIADO PEDE

Mostrar a string repetindo cada letra tantas vezes quanto o seu índice alfabético: o `a` sai uma vez, o `b` duas, o `e` cinco, e assim por diante. A caixa não muda e o que não for letra sai uma vez. No fim, um newline.

Se o número de argumentos não for 1, mostrar só o newline.

Funções permitidas: `write`.

```
$> ./repeat_alpha "abacadaba 42!" | cat -e  
abbacccaddddabba 42!$
```

COMO PENSAR

- 1 O ciclo de sempre, mas agora cada volta escreve um número variável de caracteres — logo é preciso um **segundo ciclo lá dentro**.
- 2 Quantas vezes? Para as letras, a posição no alfabeto mais um (`a` é a posição 0, mas sai uma vez). Para tudo o resto, uma.
- 3 O truque de organização: começar por assumir o caso normal (`n = 1`) e só depois corrigir se for letra. Assim os espaços e os números ficam tratados sem escrever nada para eles.

O CÓDIGO

repeat_alpha.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  int     main(int argc, char **argv)
9  {
10     int    i;
11     int    j;
12     int    n;
13     char   c;
14
15     if (argc == 2)
16     {
17         i = 0;
18         while (argv[1][i])
19         {
20             c = argv[1][i];
21             n = 1;
22             if (c >= 'a' && c <= 'z')
23                 n = c - 'a' + 1;
24             else if (c >= 'A' && c <= 'Z')
25                 n = c - 'A' + 1;
26             j = 0;
27             while (j++ < n)
28                 ft_putchar(c);
29             i++;
30         }
31     }
32     ft_putchar('\n');
33     return (0);
34 }
```

LINHA A LINHA

- 10–13** Quatro variáveis: o índice de fora, o contador de dentro, quantas vezes repetir, e o caractere actual.
- 21** O caso normal: uma vez. Tudo o que não for letra fica com este valor.
- 22–23** Se é minúscula, quantas vezes é a posição no alfabeto mais um: o **a** dá 1, o **b** dá 2.
- 24–25** O mesmo nas maiúsculas.
- 26–28** O ciclo de dentro: escreve o caractere **n** vezes. O **j++ < n** compara e só depois incrementa, por isso corre exactamente **n** voltas.
- 29** Passa ao caractere seguinte da string — este **i++** pertence ao ciclo de fora.

N1 search_and_replace

Substituir uma letra por outra dentro de uma string.

O QUE O ENUNCIADO PEDE

O programa recebe três argumentos: uma string, a letra a procurar e a letra a pôr no lugar. Mostra a string com todas as ocorrências substituídas, seguida de um newline.

Se o número de argumentos não for 3, mostra só o newline. O segundo e o terceiro argumentos têm de ser **uma letra só** — se tiverem mais, também sai só o newline.

Funções permitidas: **write**, **exit**.

```
$> ./search_and_replace "Papache est un sabre" "a" "o" | cat -e
Popoche est un sobre$
$> ./search_and_replace "zaz" "art" "zul" | cat -e
$
$> ./search_and_replace "zaz" "r" "u" | cat -e
zaz$
```

COMO PENSAR

- 1 A substituição em si é trivial: percorrer a string e, se o caractere for igual ao procurado, escrever o outro.
- 2 A dificuldade está na **validação**. O segundo exemplo só dá newline porque "art" tem três letras, e é fácil não reparar nisso.
- 3 Para saber se um argumento tem uma letra só, mede-se o comprimento — daí o `ft_strlen` ali dentro.
- 4 Se a letra procurada não existir na string, não é preciso fazer nada de especial: o ciclo escreve tudo igual e o resultado sai certo sozinho.

O CÓDIGO

search_and_replace.c

```
1 #include <unistd.h>
2
3 void ft_putchar(char c)
4 {
5     write(1, &c, 1);
6 }
7
8 int ft_strlen(char *s)
9 {
10     int i;
11
12     i = 0;
13     while (s[i])
14         i++;
15     return (i);
16 }
17
18 int main(int argc, char **argv)
19 {
20     int i;
21
22     if (argc == 4 && ft_strlen(argv[2]) == 1 && ft_strlen(argv[3]) == 1)
23     {
24         i = 0;
25         while (argv[1][i])
26         {
27             if (argv[1][i] == argv[2][0])
28                 ft_putchar(argv[3][0]);
29             else
30                 ft_putchar(argv[1][i]);
31             i++;
32         }
33     }
34     ft_putchar('\n');
35     return (0);
36 }
```

LINHA A LINHA

- 8-16 O `ft_strlen` do exercício anterior, copiado para aqui. Serve só para validar os argumentos.
- 22 As três condições de uma vez: três argumentos (logo `argc == 4`), e o segundo e o terceiro com uma letra só.
- 25 O ciclo percorre a **primeira** string, que é a que vai ser mostrada.
- 27 `argv[2][0]` é o primeiro (e único) caractere do segundo argumento — a letra a procurar.
- 28 Se bateu certo, escreve-se a letra de substituição em vez da original.
- 30 Senão, escreve-se o caractere tal como estava.
- 34 O newline fora do `if`, como sempre: cobre todos os casos inválidos de uma vez.

O enunciado permite o `exit`, mas não é preciso nenhum: pôr o newline fora do `if` já resolve todos os casos de erro.

N1 first_word

Mostrar a primeira palavra da string.

O QUE O ENUNCIADO PEDE

Mostrar a primeira palavra da string, seguida de um newline. Uma palavra é um pedaço de string delimitado por espaços ou tabs, ou pelo princípio e fim da string.

Se o número de parâmetros não for 1, ou se não houver palavra nenhuma, mostrar só o newline.

Funções permitidas: `write`.

```
$> ./first_word " lorem,ipsum " | cat -e
lorem,ipsum$
$> ./first_word " " | cat -e
$
$> ./first_word "a" "b" | cat -e
$
```

COMO PENSAR

- 1 Começa aqui a família dos exercícios de palavras, que são seis na pool. Todos dependem da mesma pergunta: "este caractere é um separador?". Vale a pena escrever isso numa função à parte.
- 2 O esquema é sempre o mesmo: **saltar os separadores**, depois **tratar a palavra**.
- 3 Se a string for só espaços, o primeiro ciclo chega ao `'\0'` e o segundo não escreve nada — o caso sai resolvido de graça.

O CÓDIGO

first_word.c

```
1 #include <unistd.h>
2
3 void ft_putchar(char c)
4 {
5     write(1, &c, 1);
6 }
7
8 int is_space(char c)
9 {
10     return (c == ' ' || c == '\t');
11 }
12
13 int main(int argc, char **argv)
14 {
15     int i;
16
17     if (argc == 2)
18     {
19         i = 0;
20         while (is_space(argv[1][i]))
21             i++;
22         while (argv[1][i] && !is_space(argv[1][i]))
23         {
24             ft_putchar(argv[1][i]);
25             i++;
26         }
27     }
28     ft_putchar('\n');
29     return (0);
30 }
```

LINHA A LINHA

- 8-11** O ajudante. Devolve verdadeiro se o caractere for espaço ou tab. **O tab conta** — quem trata só o espaço passa nos exemplos visíveis e falha no teste do corretor.
- 20-21** Primeiro ciclo: salta os separadores do início. Repara que não precisa de testar o `'\0'`, porque o `'\0'` não é espaço nem tab e o ciclo pára nele à mesma.
- 22** Segundo ciclo: escreve enquanto houver caracteres e não for separador. As duas condições são precisas: uma pára no fim da string, a outra no fim da palavra.
- 28** O newline fora do `if`.

N1 fizzbuzz

Os números de 1 a 100, com `fizz`, `buzz` e `fizzbuzz`.

O QUE O ENUNCIADO PEDE

Mostrar os números de 1 a 100, um por linha. Se o número for múltiplo de 3, escrever `fizz` em vez dele; se for múltiplo de 5, `buzz`; se for múltiplo dos dois, `fizzbuzz`.

Funções permitidas: `write`.

```
$> ./fizzbuzz | head -6
1
2
fizz
4
buzz
fizz
```

COMO PENSAR

- ❶ **A ordem dos testes é o exercício todo.** O caso duplo tem de ser testado primeiro: se se testar o múltiplo de 3 antes, o 15 sai como `fizz` e o programa falha.
- ❷ "Múltiplo de 3 e de 5 ao mesmo tempo" é o mesmo que "múltiplo de 15".
- ❸ Não há `printf`, por isso é preciso a função que escreve números — a `ft_putnbr` das bases.

O CÓDIGO

fizzbuzz.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  void    ft_putnbr(int n)
9  {
10     if (n >= 10)
11         ft_putnbr(n / 10);
12     ft_putchar(n % 10 + '0');
13 }
14
15 int     main(void)
16 {
17     int i;
18
19     i = 1;
20     while (i <= 100)
21     {
22         if (i % 15 == 0)
23             write(1, "fizzbuzz", 8);
24         else if (i % 3 == 0)
25             write(1, "fizz", 4);
26         else if (i % 5 == 0)
27             write(1, "buzz", 4);
28         else
29             ft_putnbr(i);
30         ft_putchar('\n');
31         i++;
32     }
33     return (0);
34 }
```

LINHA A LINHA

- 8-13 A `ft_putnbr`. É recursiva: com 42, chama-se a si própria com 4, essa escreve o `'4'`, e ao regressar escreve o `'2'`. Assim os algarismos saem pela ordem certa.
-
- 12 `n % 10` é o último algarismo, e somar `'0'` transforma o número no caractere correspondente.
-
- 22 O caso duplo, testado **primeiro**. É a linha que decide se o exercício passa.
-
- 24-27 Os dois casos simples, por qualquer ordem entre si.
-
- 29 Só chega aqui quem não é múltiplo de 3 nem de 5.
-
- 30 O newline depois de cada linha, seja qual for o caso.

N2 Nível 2

15 exercícios. Comparações entre duas strings, bits, e a conversão de texto para número.

N2 alpha_mirror

Cada letra troca pela simétrica do alfabeto.

O QUE O ENUNCIADO PEDE

Mostrar a string com cada letra substituída pela letra oposta do alfabeto: o **a** passa a **z**, o **b** a **y**, o **M** a **N**. A caixa não muda e o que não for letra fica igual. No fim, um newline.

Se o número de argumentos não for 1, mostrar só o newline.

Funções permitidas: `write`.

```
$> ./alpha_mirror "My horse is Amazing." | cat -e
Nb slihv rh ZnZarMt.$
```

COMO PENSAR

- 1 Outra vez o mesmo ciclo do `ulstr`, com uma decisão diferente lá dentro.
- 2 A ideia: `c - 'a'` diz a que distância a letra está do princípio do alfabeto. A letra simétrica está **à mesma distância, mas contada a partir do fim**.
- 3 Não é preciso saber que o alfabeto tem 26 letras nem que o **a** é o primeiro: a subtração trata disso sozinha.

O CÓDIGO

alpha_mirror.c

```
1 #include <unistd.h>
2
3 void ft_putchar(char c)
4 {
5     write(1, &c, 1);
6 }
7
8 int main(int argc, char **argv)
9 {
10     int i;
11     char c;
12
13     if (argc == 2)
14     {
15         i = 0;
16         while (argv[1][i])
17         {
18             c = argv[1][i];
19             if (c >= 'a' && c <= 'z')
20                 c = 'z' - (c - 'a');
21             else if (c >= 'A' && c <= 'Z')
22                 c = 'Z' - (c - 'A');
23             ft_putchar(c);
24             i++;
25         }
26     }
27     ft_putchar('\n');
28     return (0);
29 }
```

LINHA A LINHA

- 20 `c = 'a'` é a distância ao `a`. `'z' -` essa distância dá a letra à mesma distância do `z`. Com `c = 'a'` dá `'z'`; com `c = 'b'` dá `'y'`.
- 22 O mesmo nas maiúsculas, a partir do `'Z'`.

N2 ft_strrev

Função que inverte uma string no próprio sítio.

O QUE O ENUNCIADO PEDE

Escrever uma função que inverte uma string **no sítio** e devolve o parâmetro que recebeu.

A declaração é `char *ft_strrev(char *str);`

"No sítio" quer dizer que não se pode criar uma string nova: tem de se mexer nesta.

COMO PENSAR

- 1 Se não se pode criar espaço novo, a única forma é **trocar pares de caracteres**: o primeiro com o último, o segundo com o penúltimo, e por aí fora.
- 2 Dois índices, um em cada ponta, que se aproximam um do outro.
- 3 Quando é que param? Quando se cruzam. Se continuassem, começavam a desfazer as trocas que já tinham feito.
- 4 Cada troca precisa de uma variável temporária, tal como no `ft_swap`.

O CÓDIGO

ft_strrev.c

```
1 char *ft_strrev(char *str)
2 {
3     int i;
4     int j;
5     char tmp;
6
7     i = 0;
8     j = 0;
9     while (str[j])
10        j++;
11    j--;
12    while (i < j)
13    {
14        tmp = str[i];
15        str[i] = str[j];
16        str[j] = tmp;
17        i++;
18        j--;
19    }
20    return (str);
21 }
```

LINHA A LINHA

- 9–10 Mede a string: o `j` acaba em cima do `'\0'`.
- 11 Recua um, para o `j` ficar no último caractere verdadeiro.
- 12 Enquanto os dois índices não se cruzarem. Se a string tiver um número ímpar de caracteres, o do meio fica onde está — e está certo.
- 14–16 A troca clássica em três passos, com o `tmp` a guardar um dos valores.
- 17–18 As duas pontas aproximam-se.
- 20 Devolve o mesmo ponteiro que recebeu, como o enunciado pede.

N2 ft_strcmp

Função que compara duas strings.

O QUE O ENUNCIADO PEDE

Reproduzir o comportamento do `strcmp` do `man`: devolve 0 se as strings forem iguais, um número negativo se a primeira vier antes na ordem alfabética, e positivo se vier depois.

A declaração é `int ft_strcmp(char *s1, char *s2);`

COMO PENSAR

- 1 Avançar enquanto os caracteres forem iguais. Onde o ciclo parar, está a primeira diferença.
- 2 Se as strings forem iguais até ao fim, o ciclo pára no `'\0'` das duas, e a diferença dá zero — o resultado certo, sem código extra.
- 3 O sinal sai da subtração: se o caractere do `s1` for menor, a subtração dá negativo.

O CÓDIGO

ft_strcmp.c

```
1 int ft_strcmp(char *s1, char *s2)
2 {
3     int i;
4
5     i = 0;
6     while (s1[i] && s1[i] == s2[i])
7         i++;
8     return ((unsigned char)s1[i] - (unsigned char)s2[i]);
9 }
```

LINHA A LINHA

- 6 Duas condições ao mesmo tempo: não chegou ao fim do `s1` e os caracteres são iguais. Basta uma falhar para o ciclo parar.
- 8 A subtração na posição onde pararam. O `(unsigned char)` é para caracteres acentuados ou com o bit alto: sem ele, um caractere com código acima de 127 seria lido como negativo e o sinal saía trocado.

Esta versão devolve a diferença real entre os dois caracteres, por exemplo `-99`. O `strcmp` do sistema pode devolver `-1`. Não faz mal: o que se compara é o **sinal**, não o valor.

N2 max

Função que devolve o maior inteiro de um array.

O QUE O ENUNCIADO PEDE

Escrever a função `int max(int* tab, unsigned int len);`

O primeiro parâmetro é um array de inteiros e o segundo é quantos elementos ele tem. A função devolve o maior. Se o array estiver vazio, devolve 0.

COMO PENSAR

- 1 Guardar um "melhor até agora" e comparar com cada elemento.
- 2 **Com que valor começar?** Não pode ser 0: um array só com números negativos devolveria 0, que nem sequer está lá dentro. Começa-se com o primeiro elemento do array.
- 3 Como o primeiro já foi usado, o ciclo começa no segundo.
- 4 O caso do array vazio tem de ser tratado antes de tudo, senão `tab[0]` lê memória que não existe.

O CÓDIGO

max.c

```
1 int    max(int *tab, unsigned int len)
2 {
3     unsigned int    i;
4     int             best;
5
6     if (len == 0)
7         return (0);
8     best = tab[0];
9     i = 1;
10    while (i < len)
11    {
12        if (tab[i] > best)
13            best = tab[i];
14        i++;
15    }
16    return (best);
17 }
```

LINHA A LINHA

- | | |
|-------|---|
| 3 | O <code>i</code> é declarado <code>unsigned int</code> de propósito, igual ao <code>len</code> . Comparar um <code>int</code> com um <code>unsigned int</code> dá aviso do compilador, e aviso é exercício falhado. |
| 6-7 | O caso do array vazio, tratado primeiro para o <code>tab[0]</code> da linha seguinte ser seguro. |
| 8 | O candidato começa no primeiro elemento, não em zero. |
| 9 | O ciclo começa no segundo elemento, porque o primeiro já é o candidato. |
| 12-13 | Se encontrou um maior, passa a ser o novo candidato. |
| 16 | No fim, o candidato é o maior de todos. |

N2 ft_strdup

Função que devolve uma cópia nova de uma string.

O QUE O ENUNCIADO PEDE

Reproduzir o comportamento do `strdup` do `man`: devolve uma cópia nova da string recebida.

A declaração é `char *ft_strdup(char *src);`

Funções permitidas: `malloc`.

COMO PENSAR

- 1 Aqui é preciso devolver uma string que ainda não existe. Uma variável normal declarada dentro da função desaparece quando a função acaba, por isso é preciso pedir memória com `malloc`.
- 2 O padrão do `malloc` tem sempre três passos, por esta ordem: **contar**, **pedir**, **encher**.

- 3 No passo de contar, o número é o comprimento **mais um**, para caber o `'\0'` no fim. Esquecer esse `+ 1` é o erro mais comum de todo o nível 2.

O CÓDIGO

ft_strdup.c

```
1 #include <stdlib.h>
2
3 char *ft_strdup(char *src)
4 {
5     char *dup;
6     int i;
7
8     i = 0;
9     while (src[i])
10         i++;
11     dup = malloc(sizeof(char) * (i + 1));
12     if (!dup)
13         return (0);
14     i = 0;
15     while (src[i])
16     {
17         dup[i] = src[i];
18         i++;
19     }
20     dup[i] = '\0';
21     return (dup);
22 }
```

LINHA A LINHA

1	O <code>malloc</code> vive no <code>stdlib.h</code> .
8-10	Passo 1, contar: mede a string de origem.
11	Passo 2, pedir. <code>sizeof(char)</code> é quanto espaço ocupa um caractere, e multiplica-se por quantos são precisos. O <code>+ 1</code> é o espaço do <code>'\0'</code> .
12-13	Se o <code>malloc</code> falhar devolve <code>0</code> , e é preciso devolver <code>0</code> também em vez de continuar a escrever em memória que não existe.
14	Reaproveita-se o <code>i</code> , agora a partir do zero outra vez.
15-19	Passo 3, encher: copia caractere a caractere.
20	O ciclo parou no <code>'\0'</code> sem o copiar, tal como no <code>ft_strcpy</code> . Põe-se à mão.
21	Devolve o endereço da cópia nova.

N2 ft_atoi

Função que converte texto em número inteiro.

O QUE O ENUNCIADO PEDE

Escrever uma função que converte a string recebida num inteiro e o devolve, como o `atoi` do `man`.

A declaração é `int ft_atoi(const char *str);`

O `atoi` salta os espaços do início, lê um sinal se existir, lê os algarismos e pára no primeiro caractere que não seja algarismo.

COMO PENSAR

- 1 A receita para transformar texto em número é sempre a mesma: começa-se em 0 e, por cada algarismo, **multiplica-se o que já se tem por 10 e soma-se o novo**. Com `"123"`: $0 \times 10 + 1 = 1$; $1 \times 10 + 2 = 12$; $12 \times 10 + 3 = 123$.

- 2 O trabalho divide-se em três fases claras, e escrever um ciclo para cada uma é o que torna isto fácil: saltar espaços, ler sinais, ler algarismos.
- 3 `c - '0'` transforma o caractere `'7'` no número 7, porque os algarismos são códigos seguidos.

O CÓDIGO

ft_atoi.c

```
1 int ft_atoi(const char *str)
2 {
3     int i;
4     int sign;
5     int res;
6
7     i = 0;
8     sign = 1;
9     res = 0;
10    while (str[i] == ' ' || (str[i] >= 9 && str[i] <= 13))
11        i++;
12    while (str[i] == '+' || str[i] == '-')
13    {
14        if (str[i] == '-')
15            sign = -sign;
16        i++;
17    }
18    while (str[i] >= '0' && str[i] <= '9')
19    {
20        res = res * 10 + (str[i] - '0');
21        i++;
22    }
23    return (res * sign);
24 }
```

LINHA A LINHA

- 8 O sinal começa positivo. No fim multiplica-se o resultado por ele.
- 10 Fase 1: saltar espaços. Os "espaços" do `atoi` incluem o tab e o newline, que são os códigos de 9 a 13 — daí este segundo teste.
- 12-17 Fase 2: os sinais. Cada `-` inverte o sinal acumulado, por isso `--42` dá 42.
- 18 Fase 3: os algarismos. O ciclo pára sozinho no primeiro caractere que não seja um algarismo, e é por isso que `"-42abc"` dá `-42`.
- 20 A receita do vezes dez. Os parênteses não são precisos, mas ajudam a ler.
- 23 Aplica o sinal só no fim.

Esta versão aceita vários sinais seguidos, por tradição da Piscine. O `atoi` verdadeiro do `man` só aceita um — se for preciso ser fiel ao `man`, troca-se o `while` da linha 12 por um `if`.

N2 do_op

Fazer uma conta com dois números e um operador.

O QUE O ENUNCIADO PEDE

O programa recebe três argumentos: um número, um operador (`+`, `-`, `*`, `/` ou `%`) e outro número. Mostra o resultado da conta seguido de um newline.

Se o número de parâmetros não for 3, mostra só o newline.

Funções permitidas: `atoi`, `printf`, `write`.

```
$> ./do_op "1" "+" "-43" | cat -e
-42$
$> ./do_op "9828" "/" 234 | cat -e
42$
```

COMO PENSAR

- 1 **Olhar primeiro para a linha** `Allowed functions`. Este exercício permite o `atoi` e o `printf` verdadeiros, o que elimina metade do trabalho. É dos mais rápidos do nível 2.
- 2 Converter os dois números, ver qual é o operador, fazer a conta.
- 3 O operador chega como uma **string de um caractere**. Compara-se `argv[2][0]`, que é o caractere, e não `argv[2]`, que é a string.

O CÓDIGO

do_op.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(int argc, char **argv)
5 {
6     int a;
7     int b;
8
9     if (argc != 4)
10    {
11        printf("\n");
12        return (0);
13    }
14    a = atoi(argv[1]);
15    b = atoi(argv[3]);
16    if (argv[2][0] == '+')
17        printf("%d\n", a + b);
18    else if (argv[2][0] == '-')
19        printf("%d\n", a - b);
20    else if (argv[2][0] == '*')
21        printf("%d\n", a * b);
22    else if (argv[2][0] == '/')
23        printf("%d\n", a / b);
24    else if (argv[2][0] == '%')
25        printf("%d\n", a % b);
26    return (0);
27 }
```

LINHA A LINHA

9-13 O caso dos argumentos errados, resolvido logo e o programa acaba ali.

14-15 O `atoi` faz o trabalho do exercício anterior de graça. Repara que o segundo número é o `argv[3]`, porque o `argv[2]` é o operador.

16 `argv[2][0]` é o primeiro caractere do segundo argumento. `'+'` com plicas é um caractere; `"+"` com aspas seria uma string, e em C não se comparam strings com `==`.

17 `%d` no `printf` significa "escreve aqui um inteiro". O `\n` é o newline que o enunciado pede.

Algumas versões deste enunciado pedem `Stop : division by zero` e `Stop : modulo by zero` quando o segundo número é zero. **Lê o teu enunciado.** Se pedir, acrescenta antes das divisões um teste a `b == 0` que escreve a mensagem e faz `return`.

O QUE O ENUNCIADO PEDE

Mostrar a última palavra da string, seguida de um newline. Uma palavra é um pedaço de string delimitado por espaços ou tabs, ou pelo princípio e fim da string.

Se o número de parâmetros não for 1, ou se não houver palavras, mostrar só o newline.

Funções permitidas: `write`.

```
$> ./last_word "this      ...      is sparta, then again, maybe    not" | cat -e
not$
$> ./last_word "  lorem,ipsum  " | cat -e
lorem,ipsum$
```

COMO PENSAR

- 1 É o `first_word` ao contrário, mas com uma dificuldade a mais: para escrever a palavra é preciso saber onde ela **começa** e onde **acaba**, e andando de trás para a frente descobre-se primeiro o fim.
- 2 São quatro passos: ir ao fim da string; recuar enquanto for separador (salta os espaços finais); marcar aí o fim da palavra; continuar a recuar enquanto **não** for separador.
- 3 Quando esse último ciclo pára, o índice está um caractere **antes** da palavra. Por isso avança-se um antes de escrever.
- 4 O teste `i >= 0` em todos os ciclos é o que evita ler fora da string quando ela é vazia ou só tem espaços.

O CÓDIGO

last_word.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  int     is_space(char c)
9  {
10     return (c == ' ' || c == '\t');
11 }
12
13 int     main(int argc, char **argv)
14 {
15     int i;
16     int end;
17
18     if (argc == 2)
19     {
20         i = 0;
21         while (argv[1][i])
22             i++;
23         i--;
24         while (i >= 0 && is_space(argv[1][i]))
25             i--;
26         end = i;
27         while (i >= 0 && !is_space(argv[1][i]))
28             i--;
29         i++;
30         while (i <= end)
31         {
32             ft_putchar(argv[1][i]);
33             i++;
34         }
35     }
36     ft_putchar('\n');
37     return (0);
38 }
```

LINHA A LINHA

- | | |
|-------|---|
| 21-23 | Vai ao fim e recua um, para ficar no último caractere verdadeiro. |
| 24-25 | Salta os separadores do fim. Aqui é que o caso " lorem, ipsum " se resolve. |
| 26 | Marca o fim da palavra. |
| 27-28 | Recua enquanto não for separador: está a passar por cima da palavra. |
| 29 | Avança um. Agora o i está no primeiro caractere da palavra. |
| 30-34 | Escreve do princípio ao fim da palavra, agora para a frente. |

Se a string for vazia, o **i** acaba em **-1** logo na linha 23, todos os ciclos são saltados, o **end** fica **-1** e o ciclo de escrita nunca corre. Sai só o newline, que é o que o enunciado pede.

O QUE O ENUNCIADO PEDE

Mostrar os caracteres que aparecem nas duas strings, sem repetições, pela ordem em que aparecem na primeira. No fim, um newline.

Se o número de argumentos não for 2, mostrar só o newline.

Funções permitidas: `write`.

```
$> ./inter "padinton" "paqefwtdjetyiytjneytjoeyjnejejj" | cat -e
padinto$
```

COMO PENSAR

- 1 Duas perguntas por cada caractere da primeira string: **existe na segunda?** e **já escrevi este antes?**
- 2 A primeira resolve-se com um ciclo que procura na segunda string.
- 3 A segunda resolve-se com uma ideia nova: um array de 256 marcadores, um por cada caractere possível. Como cada caractere é um número entre 0 e 255, pode usar-se o próprio caractere como **posição** no array. A gaveta do `a` é `seen['a']`.
- 4 Marca-se 1 quando se escreve, e antes de escrever verifica-se se a gaveta já está marcada.

O CÓDIGO`inter.c`

```
1 #include <unistd.h>
2
3 void    ft_putchar(char c)
4 {
5     write(1, &c, 1);
6 }
7
8 int     main(int argc, char **argv)
9 {
10     int    i;
11     int    j;
12     char    seen[256];
13
14     i = 0;
15     while (i < 256)
16         seen[i++] = 0;
17     if (argc == 3)
18     {
19         i = 0;
20         while (argv[1][i])
21         {
22             j = 0;
23             while (argv[2][j] && argv[2][j] != argv[1][i])
24                 j++;
25             if (argv[2][j] && !seen[(unsigned char)argv[1][i]])
26             {
27                 ft_putchar(argv[1][i]);
28                 seen[(unsigned char)argv[1][i]] = 1;
29             }
30             i++;
31         }
32     }
33     ft_putchar('\n');
34     return (0);
35 }
```

LINHA A LINHA

- 12 O array de marcadores. 256 porque é quantos caracteres diferentes cabem num byte.
- 14-16 Põe tudo a zero. Um array acabado de criar tem lixo lá dentro, não zeros.
- 22-24 Procura o caractere na segunda string. O ciclo pára ou em cima dele (encontrou) ou no `'\0'` (não encontrou).
- 25 Duas condições: `argv[2][j]` ainda existe, logo encontrou; e a gaveta ainda não está marcada, logo é a primeira vez.
- 27-28 Escreve e marca a gaveta, para não voltar a escrever o mesmo caractere.

O `(unsigned char)` à volta do índice é obrigatório. Sem ele, um caractere acentuado seria lido como número negativo, e `seen[-30]` lê memória fora do array — o programa rebenta.

N2 union

Os caracteres que aparecem numa string ou na outra, sem repetidos.

O QUE O ENUNCIADO PEDE

Mostrar os caracteres que aparecem numa das strings ou na outra, sem repetições, pela ordem em que aparecem na linha de comando. No fim, um newline.

Se o número de argumentos não for 2, mostrar só o newline.

Funções permitidas: `write`.

```
$> ./union zpadinton "paqefwtdjetyiytjneytjoeyjnejejj" | cat -e
zpadintoqefwjy$
```

COMO PENSAR

- 1 É o `inter` sem a busca na segunda string, mas com um ciclo por fora que passa pelas duas.
- 2 Os marcadores fazem todo o trabalho: escreve-se qualquer caractere que ainda não tenha sido escrito, venha ele de onde vier.
- 3 Vale a pena treinar este e o `inter` no mesmo dia: quem resolve um tem o outro quase de graça.

O CÓDIGO

union.c

```
1 #include <unistd.h>
2
3 void    ft_putchar(char c)
4 {
5     write(1, &c, 1);
6 }
7
8 int     main(int argc, char **argv)
9 {
10     int    a;
11     int    i;
12     char    seen[256];
13
14     i = 0;
15     while (i < 256)
16         seen[i++] = 0;
17     if (argc == 3)
18     {
19         a = 1;
20         while (a <= 2)
21         {
22             i = 0;
23             while (argv[a][i])
24             {
25                 if (!seen[(unsigned char)argv[a][i]])
26                 {
27                     ft_putchar(argv[a][i]);
28                     seen[(unsigned char)argv[a][i]] = 1;
29                 }
30                 i++;
31             }
32             a++;
33         }
34     }
35     ft_putchar('\n');
36     return (0);
37 }
```

LINHA A LINHA

- 19-20 O ciclo novo: `a` vale 1 na primeira volta e 2 na segunda, ou seja, percorre `argv[1]` e depois `argv[2]`.
- 23 O mesmo ciclo de sempre, mas agora sobre `argv[a]`, que muda a cada volta de fora.
- 25 Só a pergunta "já escrevi este?". Não há busca nenhuma na outra string.
- 32 Passa da primeira string para a segunda.

N2 wdmatch

Ver se a primeira string se consegue escrever com os caracteres da segunda, pela ordem.

O QUE O ENUNCIADO PEDE

Verificar se é possível escrever a primeira string usando caracteres da segunda, respeitando a ordem em que aparecem na segunda. Se for possível, mostrar a primeira string seguida de um newline; senão, só o newline.

Se o número de argumentos não for 2, mostrar só o newline.

Funções permitidas: `write`.

```
$> ./wdmatch "faya" "fgvvfdxcacpolhyghbreda" | cat -e
faya$
$> ./wdmatch "faya" "fgvvfdxcacpolhyghbred" | cat -e
$
```

COMO PENSAR

- 1 Técnica nova, mas simples: **um índice em cada string**.
- 2 O índice da segunda string anda **sempre**, caractere a caractere. O da primeira só avança quando encontra o caractere que procurava.
- 3 No fim, se o índice da primeira chegou ao `'\0'`, é porque todos os caracteres foram encontrados pela ordem certa.
- 4 Não é preciso marcar nada nem procurar para trás: uma passagem por cada string chega.

O CÓDIGO

wdmatch.c

```
1 #include <unistd.h>
2
3 void ft_putchar(char c)
4 {
5     write(1, &c, 1);
6 }
7
8 int main(int argc, char **argv)
9 {
10     int i;
11     int j;
12
13     if (argc == 3)
14     {
15         i = 0;
16         j = 0;
17         while (argv[1][i] && argv[2][j])
18         {
19             if (argv[1][i] == argv[2][j])
20                 i++;
21             j++;
22         }
23         if (!argv[1][i])
24         {
25             j = 0;
26             while (argv[1][j])
27                 ft_putchar(argv[1][j++]);
28         }
29     }
30     ft_putchar('\n');
31     return (0);
32 }
```

LINHA A LINHA

- | | |
|-------|---|
| 17 | O ciclo anda enquanto houver caracteres nas duas. Basta uma acabar para parar. |
| 19-20 | Só quando bate certo é que o índice da primeira string avança. |
| 21 | O da segunda avança sempre, tenha batido certo ou não. |
| 23 | <code>!argv[1][i]</code> quer dizer "o <code>i</code> está em cima do <code>'\0'</code> ", ou seja, percorreu a primeira string toda. |
| 25-27 | Escreve a primeira string. O <code>j++</code> dentro dos parênteses usa o valor e só depois incrementa. |

N2 is_power_of_2

Função que diz se um número é potência de 2.

O QUE O ENUNCIADO PEDE

Escrever uma função que determina se um número é potência de 2. Devolve 1 se for, 0 se não for.

A declaração é `int is_power_of_2(unsigned int n);`

COMO PENSAR

- 1 Em binário, uma potência de 2 tem **exactamente um bit a 1**: o 8 é `1000`, o 16 é `10000`.
- 2 Subtrair 1 a um número desses vira todos os bits abaixo desse: `8 - 1` dá `0111`.
- 3 Logo, o **E** bit a bit entre os dois (`n & (n - 1)`) dá zero **só** nas potências de 2.
- 4 O zero tem de ser tratado à parte: não é potência de 2, mas `0 & (0-1)` também daria zero.

O CÓDIGO

`is_power_of_2.c`

```
1 int    is_power_of_2(unsigned int n)
2 {
3     if (n == 0)
4         return (0);
5     return ((n & (n - 1)) == 0);
6 }
```

LINHA A LINHA

3-4 O caso do zero, tratado primeiro.

- 5 O teste. O `&` é o "E" bit a bit: só deixa ficar a 1 os bits que estão a 1 nos **dois** números. A comparação `== 0` devolve 1 ou 0, que é exactamente o que o enunciado pede.

Se esta ideia não vier à cabeça no exame, a versão à bruta também passa e escreve-se em três linhas: dividir por 2 enquanto o número for par, e no fim verificar se sobrou 1. Não vale a pena perder tempo a tentar ser esperto.

N2 print_bits

Função que escreve um byte em binário.

O QUE O ENUNCIADO PEDE

Escrever uma função que recebe um byte e o mostra em binário, **sem newline no fim**.

A declaração é `void print_bits(unsigned char octet);`

Por exemplo, o valor 2 escreve `00000010`.

Funções permitidas: `write`.

COMO PENSAR

- 1 Um `unsigned char` são 8 bits. É preciso escrever os 8, do mais à esquerda para o mais à direita.
- 2 Para ver o bit número `i` há uma receita única que serve sempre: `(octet >> i) & 1`. O `>>` empurra os bits `i` casas para a direita, o que põe o bit que interessa na ponta; o `& 1` apaga tudo o resto.
- 3 Como se escreve da esquerda para a direita, começa-se no bit 7 e desce-se até ao 0.
- 4 Um bit vale 0 ou 1, que são **números**. Somar `'0'` transforma-os nos caracteres `'0'` e `'1'`.

O CÓDIGO

print_bits.c

```
1 #include <unistd.h>
2
3 void print_bits(unsigned char octet)
4 {
5     int i;
6     char c;
7
8     i = 7;
9     while (i >= 0)
10    {
11        c = ((octet >> i) & 1) + '0';
12        write(1, &c, 1);
13        i--;
14    }
15 }
```

LINHA A LINHA

- 8 Começa no bit mais à esquerda, que é o número 7.
- 11 A receita: empurra `i` casas, fica com o último bit, e soma `'0'` para virar caractere.
- 12 Escreve esse caractere.
- 13 Desce um bit.

O enunciado é explícito: **sem newline no fim**. Acrescentar um é falhar o exercício.

N2 reverse_bits

Função que inverte a ordem dos bits de um byte.

O QUE O ENUNCIADO PEDE

Escrever uma função que recebe um byte, inverte-o bit a bit e devolve o resultado. Por exemplo, `0100 0001` passa a `1000 0010`.

A declaração é `unsigned char reverse_bits(unsigned char octet);`

COMO PENSAR

- ❶ Constrói-se o resultado a partir do zero, um bit de cada vez.
- ❷ A cada volta: empurrar o que já se tem uma casa para a esquerda (para abrir espaço à direita) e meter aí o bit novo.
- ❸ Como os bits do original se leem do 0 para cima e se arrumam sempre na ponta direita, saem naturalmente pela ordem inversa — que é exactamente o que se pede.

O CÓDIGO

reverse_bits.c

```
1 unsigned char reverse_bits(unsigned char octet)
2 {
3     unsigned char res;
4     int i;
5
6     res = 0;
7     i = 0;
8     while (i < 8)
9     {
10         res = (res << 1) | ((octet >> i) & 1);
11         i++;
12     }
13     return (res);
14 }
```

LINHA A LINHA

6 O resultado começa vazio.

10 Duas coisas numa linha: `res << 1` abre espaço à direita, e o `|` (o "OU" bit a bit) mete lá o bit lido do original. O `((octet >> i) & 1)` é a mesma receita do `print_bits`.

13 Ao fim de oito voltas, o `res` tem os bits todos pela ordem contrária.

N2 swap_bits

Função que troca as duas metades de um byte.

O QUE O ENUNCIADO PEDE

Escrever uma função que recebe um byte, troca as suas duas metades e devolve o resultado. Por exemplo, `0100 0001` passa a `0001 0100`.

A declaração é `unsigned char swap_bits(unsigned char octet);`

COMO PENSAR

- 1 Um byte tem 8 bits, logo cada metade tem 4.
- 2 A metade da esquerda tem de descer 4 casas e a da direita subir 4.
- 3 Depois junta-se tudo com o `|`.
- 4 Não é preciso apagar nada: o que sai fora dos 8 bits é cortado automaticamente porque o tipo de retorno é `unsigned char`.

O CÓDIGO

swap_bits.c

```
1 unsigned char swap_bits(unsigned char octet)
2 {
3     return ((octet >> 4) | (octet << 4));
4 }
```

LINHA A LINHA

3 Uma linha só. `octet >> 4` desce a metade de cima; `octet << 4` sobe a de baixo; o `|` junta as duas.

N3 Nível 3

15 exercícios. Palavras, bases, matemática, a primeira memória pedida com `malloc` e a primeira lista ligada.

N3 paramsum

Mostrar quantos argumentos o programa recebeu.

O QUE O ENUNCIADO PEDE

Mostrar o número de argumentos passados ao programa, seguido de um newline. Sem argumentos, mostrar `0` e um newline.

Funções permitidas: `write`.

```
$> ./paramsum 1 2 3 5 7 24 | cat -e
6$
$> ./paramsum | cat -e
0$
```

COMO PENSAR

- 1 O `argc` já é a resposta: basta tirar 1, que é o nome do programa.
- 2 Não há `printf`, logo é preciso a `ft_putnbr` das bases.
- 3 Está no nível 3 e é dos mais fáceis do exame inteiro. Não julgues um exercício pelo nível.

O CÓDIGO

paramsum.c

```
1 #include <unistd.h>
2
3 void ft_putchar(char c)
4 {
5     write(1, &c, 1);
6 }
7
8 void ft_putnbr(int n)
9 {
10     if (n >= 10)
11         ft_putnbr(n / 10);
12     ft_putchar(n % 10 + '0');
13 }
14
15 int main(int argc, char **argv)
16 {
17     (void)argv;
18     ft_putnbr(argc - 1);
19     ft_putchar('\n');
20     return (0);
21 }
```

LINHA A LINHA

8-13 A `ft_putnbr` recursiva de sempre.

17 `(void)argv;` é obrigatório. O `argv` foi recebido e não é usado, e o `-Wextra -Werror` transforma esse aviso em erro de compilação. **Sem esta linha o programa não compila.**

18 `argc - 1` tira o nome do programa da contagem.

O QUE O ENUNCIADO PEDE

Receber um número positivo (ou zero) em base 10 como texto e mostrá-lo em base 16 com letras minúsculas, seguido de um newline.

Se o número de parâmetros não for 1, mostrar só o newline.

Funções permitidas: `write`.

```
$> ./print_hex "255" | cat -e
ff$
$> ./print_hex "5156454" | cat -e
4eae66$
```

COMO PENSAR

- 1 São duas metades independentes: transformar o **texto** num número, e depois transformar esse número em **texto** noutra base.
- 2 A primeira metade é a receita do vezes dez, igual ao `ft_atoi`.
- 3 A segunda é a `ft_putnbr` com 16 em vez de 10.
- 4 O problema de escrever em base 16 é que os restos vão de 0 a 15, e de 10 para cima já não são algarismos. A solução evita `if` nenhum: guarda-se a string `"0123456789abcdef"` e usa-se o resto como **posição** dentro dela.

O CÓDIGO

print_hex.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  void    put_hex(int n)
9  {
10     char    *base;
11
12     base = "0123456789abcdef";
13     if (n >= 16)
14         put_hex(n / 16);
15     ft_putchar(base[n % 16]);
16 }
17
18 int     main(int argc, char **argv)
19 {
20     int n;
21     int i;
22
23     if (argc == 2)
24     {
25         n = 0;
26         i = 0;
27         while (argv[1][i] >= '0' && argv[1][i] <= '9')
28         {
29             n = n * 10 + (argv[1][i] - '0');
30             i++;
31         }
32         put_hex(n);
33     }
34     ft_putchar('\n');
35     return (0);
36 }
```

LINHA A LINHA

- 12 A tabela dos dígitos. A posição 10 é o 'a', a 15 é o 'f'.
- 13-14 A recursão: se o número ainda tem mais de um dígito em base 16, trata-se primeiro a parte de cima.
- 15 `n % 16` dá um número entre 0 e 15, que é exactamente a posição certa dentro da tabela.
- 27-31 A outra metade: texto para número, com a receita do vezes dez.
- 34 O newline fora do `if`, como sempre.

N3 ft_atoi_base

Função que converte texto de qualquer base até 16 num inteiro.

O QUE O ENUNCIADO PEDE

Escrever uma função que converte a string recebida (numa base até 16) para um inteiro em base 10.

Os caracteres reconhecidos são `0123456789abcdef`, cortados conforme a base pedida: a base 4 reconhece `0123`, a base 16 reconhece tudo. As maiúsculas também contam. O sinal `-` só é interpretado se for o primeiro caractere.

A declaração é `int ft_atoi_base(const char *str, int str_base);`

COMO PENSAR

- 1 É o `ft_atoi` com a base como variável em vez do 10 fixo.
- 2 O que complica é saber quando parar: em base 4, um `7` já não é válido, e em base 16 um `g` também não.
- 3 A solução é pôr essa pergunta numa função à parte que devolve o valor do caractere **ou `-1` se ele não pertencer a esta base**. Assim o ciclo principal fica limpo e pára sozinho quando vier `-1`.

O CÓDIGO

ft_atoi_base.c

```
1 int    value_of(char c, int base)
2 {
3     int v;
4
5     v = -1;
6     if (c >= '0' && c <= '9')
7         v = c - '0';
8     else if (c >= 'a' && c <= 'f')
9         v = c - 'a' + 10;
10    else if (c >= 'A' && c <= 'F')
11        v = c - 'A' + 10;
12    if (v >= base)
13        return (-1);
14    return (v);
15 }
16
17 int    ft_atoi_base(const char *str, int str_base)
18 {
19     int i;
20     int sign;
21     int res;
22     int v;
23
24     i = 0;
25     sign = 1;
26     res = 0;
27     if (str[0] == '-')
28     {
29         sign = -1;
30         i = 1;
31     }
32     v = value_of(str[i], str_base);
33     while (v >= 0)
34     {
35         res = res * str_base + v;
36         i++;
37         v = value_of(str[i], str_base);
38     }
39     return (res * sign);
40 }
```

LINHA A LINHA

5	Começa a assumir que o caractere é inválido. Se nenhum dos <code>if</code> seguintes o reconhecer, fica assim.
6-7	Os algarismos valem o que parecem.
8-9	As minúsculas de <code>a</code> a <code>f</code> valem de 10 a 15. Daí o <code>+ 10</code> .
10-11	As maiúsculas valem o mesmo.
12-13	A parte que corta pela base: um <code>7</code> é um caractere válido em geral, mas não em base 4.
27-31	O sinal, só se for mesmo o primeiro caractere, como o enunciado exige.
32	Lê-se o primeiro valor antes do ciclo.
33	O ciclo corre enquanto o caractere for válido nesta base.
35	A mesma receita do <code>ft_atoi</code> , com <code>str_base</code> no lugar do 10.
37	Lê o valor seguinte no fim de cada volta, para a condição do <code>while</code> poder ser testada.

N3 tab_mult

Escrever a tabuada de um número.

O QUE O ENUNCIADO PEDE

Mostrar a tabuada do número recebido, de 1 a 9, no formato `1 x 9 = 9`.

Se não houver parâmetros, mostrar só um newline.

Funções permitidas: `write`.

```
$> ./tab_mult 9
1 x 9 = 9
2 x 9 = 18
3 x 9 = 27
```

COMO PENSAR

- 1 Não há ideia nova nenhuma: é a receita do vezes dez no início e nove voltas a escrever três números e dois pedaços de texto.
- 2 A única armadilha é o **formato**. Os espaços à volta do `x` e do `=` fazem parte do output e têm de estar exactamente como no enunciado.

O CÓDIGO

tab_mult.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  void    ft_putnbr(int n)
9  {
10     if (n >= 10)
11         ft_putnbr(n / 10);
12     ft_putchar(n % 10 + '0');
13 }
14
15 int     main(int argc, char **argv)
16 {
17     int n;
18     int i;
19
20     if (argc != 2)
21     {
22         ft_putchar('\n');
23         return (0);
24     }
25     n = 0;
26     i = 0;
27     while (argv[1][i] >= '0' && argv[1][i] <= '9')
28     {
29         n = n * 10 + (argv[1][i] - '0');
30         i++;
31     }
32     i = 1;
33     while (i <= 9)
34     {
35         ft_putnbr(i);
36         write(1, " x ", 3);
37         ft_putnbr(n);
38         write(1, " = ", 3);
39         ft_putnbr(i * n);
40         ft_putchar('\n');
41         i++;
42     }
43     return (0);
44 }
```

LINHA A LINHA

27-31 Texto para número, como no `print_hex`.

32-33 Nove voltas, de 1 a 9.

35 O multiplicador desta volta.

36 `" x "` com espaço de cada lado — três bytes. Copiar isto do enunciado em vez de escrever de cabeça.

39 O resultado, calculado na hora.

40 O newline no fim de cada linha da tabuada.

Conferir sempre com `| cat -e`: um espaço a mais ou a menos entre o `x` e os números falha o exercício, e não se vê a olho nu.

O QUE O ENUNCIADO PEDE

Receber dois números estritamente positivos como texto e mostrar o maior divisor comum, seguido de um newline.

Se o número de parâmetros não for 2, mostrar só o newline.

Funções permitidas: `printf`, `atoi`, `malloc`, `free`.

```
$> ./pgcd 42 12 | cat -e
6$
$> ./pgcd 17 3 | cat -e
1$
```

COMO PENSAR

- 1 Outra vez: **olhar primeiro para as funções permitidas**. Com `printf` e `atoi`, o exercício reduz-se ao algoritmo.
- 2 O algoritmo chama-se Euclides e vale a pena saber de cor, porque também resolve o `lcm`: troca-se `(a, b)` por `(b, resto de a por b)` até o `b` ser zero. O que sobrar no `a` é a resposta.
- 3 Não tentes por tentativa e erro a descer de `a` até 1 — é mais código, mais lento, e mais fácil de errar.

O CÓDIGO

pgcd.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(int argc, char **argv)
5 {
6     int a;
7     int b;
8     int t;
9
10    if (argc != 3)
11    {
12        printf("\n");
13        return (0);
14    }
15    a = atoi(argv[1]);
16    b = atoi(argv[2]);
17    while (b != 0)
18    {
19        t = a % b;
20        a = b;
21        b = t;
22    }
23    printf("%d\n", a);
24    return (0);
25 }
```

LINHA A LINHA

- | | |
|-------|--|
| 17 | O ciclo corre até o <code>b</code> ser zero. |
| 19 | O resto da divisão. É a única conta do algoritmo. |
| 20–21 | Os dois valores deslizam: o <code>b</code> passa a <code>a</code> , e o resto passa a <code>b</code> . |
| 23 | Quando o <code>b</code> chega a zero, o <code>a</code> é o maior divisor comum. |

Três linhas dentro do ciclo, e funciona para quaisquer dois números. É um dos raros sítios do exame onde decorar a receita compensa mesmo.

N3 lcm

Função que devolve o mínimo múltiplo comum.

O QUE O ENUNCIADO PEDE

Escrever uma função que recebe dois `unsigned int` e devolve o mínimo múltiplo comum. Se algum deles for zero, devolve 0.

A declaração é `unsigned int lcm(unsigned int a, unsigned int b);`

COMO PENSAR

- 1 O enunciado dá a fórmula: o mínimo múltiplo comum é o produto dos dois a dividir pelo maior divisor comum.
- 2 Ou seja, é o `pgcd` lá dentro mais uma divisão.
- 3 **A ordem das contas importa.** Multiplicar primeiro e dividir depois pode estourar o limite do `unsigned int` com números grandes. Dividir primeiro nunca estoura, porque o resultado da divisão é sempre exacto.

O CÓDIGO

`lcm.c`

```
1 unsigned int  lcm(unsigned int a, unsigned int b)
2 {
3     unsigned int  x;
4     unsigned int  y;
5     unsigned int  t;
6
7     if (a == 0 || b == 0)
8         return (0);
9     x = a;
10    y = b;
11    while (y != 0)
12    {
13        t = x % y;
14        x = y;
15        y = t;
16    }
17    return (a / x * b);
18 }
```

LINHA A LINHA

7-8 O caso do zero, que o enunciado manda tratar à parte.

9-10 Copiam-se os valores, porque o algoritmo de Euclides destrói-os e ainda são precisos no fim.

11-16 Euclides, exactamente igual ao `pgcd`. No fim, o `x` é o maior divisor comum.

17 `a / x * b` e não `a * b / x`. As duas dão o mesmo resultado em matemática, mas só a primeira é segura no computador.

N3 add_prime_sum

Somar todos os primos até ao número dado.

O QUE O ENUNCIADO PEDE

Receber um inteiro positivo e mostrar a soma de todos os números primos menores ou iguais a ele, seguida de um newline.

Se o número de argumentos não for 1, ou se o argumento não for um número positivo, mostrar `0` e um newline.

Funções permitidas: `write`, `exit`.

```
$> ./add_prime_sum 5 | cat -e
10$
$> ./add_prime_sum 7 | cat -e
17$
$> ./add_prime_sum | cat -e
0$
```

COMO PENSAR

- 1 Duas partes: saber se um número é primo, e somar os que são.
- 2 Para saber se é primo, tenta-se dividir por 2, 3, 4... Mas não é preciso ir até ao fim: **basta ir até à raiz quadrada**. Se um número tem um divisor maior que a raiz, tem obrigatoriamente outro menor, que já teria sido encontrado. Escreve-se `i * i <= n`, que evita ter de calcular raízes.
- 3 A validação do argumento é metade do exercício: o enunciado pede `0` tanto quando não há argumento como quando ele não é um número.

O CÓDIGO

add_prime_sum.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  void    ft_putnbr(int n)
9  {
10     if (n >= 10)
11         ft_putnbr(n / 10);
12     ft_putchar(n % 10 + '0');
13 }
14
15 int     is_prime(int n)
16 {
17     int i;
18
19     if (n < 2)
20         return (0);
21     i = 2;
22     while (i * i <= n)
23     {
24         if (n % i == 0)
25             return (0);
26         i++;
27     }
28     return (1);
29 }
30
31 int     main(int argc, char **argv)
32 {
33     int n;
34     int i;
35     int sum;
36
37     i = 0;
38     if (argc == 2)
39         while (argv[1][i] >= '0' && argv[1][i] <= '9')
40             i++;
41     if (argc != 2 || i == 0 || argv[1][i] != '\0')
42     {
43         write(1, "0\n", 2);
44         return (0);
45     }
46     n = 0;
47     i = 0;
48     while (argv[1][i])
49         n = n * 10 + (argv[1][i++] - '0');
50     sum = 0;
51     i = 2;
52     while (i <= n)
53     {
54         if (is_prime(i))
55             sum = sum + i;
56         i++;
57     }
58     ft_putnbr(sum);
59     ft_putchar('\n');
60     return (0);
61 }
```

LINHA A LINHA

19–20 Nem o nem 1 são primos, e os negativos também não.

22 `i * i <= n` é a mesma coisa que "`i` até à raiz de `n`", mas sem calcular raízes.

24–25 Encontrou um divisor, logo não é primo, e sai já.

28 Se nenhum divisor foi encontrado, é primo.

37–40 Conta quantos algarismos há no início do argumento.

41 Três casos de erro numa linha: não é exactamente um argumento; não começa por algarismo; ou tem alguma coisa a seguir aos algarismos.

48–49 Texto para número, com o `i++` dentro dos parênteses rectos.

52–57 Percorre todos os números de 2 até `n` e soma os que são primos.

N3 epur_str

A string com exactamente um espaço entre palavras.

O QUE O ENUNCIADO PEDE

Mostrar a string com exactamente um espaço entre palavras, sem espaços nem tabs no início nem no fim, seguida de um newline.

Se o número de argumentos não for 1, ou se não houver palavras, mostrar só o newline.

Funções permitidas: `write`.

```
$> ./epur_str " seulement la c'est plus dur " | cat -e
seulement la c'est plus dur$
```

COMO PENSAR

- 1 É o `first_word` posto dentro de um ciclo que repete: saltar separadores, escrever a palavra, outra vez.
- 2 O detalhe que decide o exercício: **o espaço separador escreve-se antes de cada palavra, excepto a primeira**. Se fosse escrito depois de cada palavra, ficava um espaço a mais no fim e o output falhava — e é esse o erro número um deste exercício.
- 3 Para saber se já foi escrita alguma palavra, usa-se uma bandeira.

O CÓDIGO

epur_str.c

```
1 #include <unistd.h>
2
3 void ft_putchar(char c)
4 {
5     write(1, &c, 1);
6 }
7
8 int is_space(char c)
9 {
10     return (c == ' ' || c == '\t');
11 }
12
13 int main(int argc, char **argv)
14 {
15     int i;
16     int first;
17
18     if (argc == 2)
19     {
20         i = 0;
21         first = 1;
22         while (argv[1][i])
23         {
24             while (is_space(argv[1][i]))
25                 i++;
26             if (argv[1][i] && !first)
27                 ft_putchar(' ');
28             while (argv[1][i] && !is_space(argv[1][i]))
29             {
30                 ft_putchar(argv[1][i]);
31                 i++;
32                 first = 0;
33             }
34         }
35     }
36     ft_putchar('\n');
37     return (0);
38 }
```

LINHA A LINHA

21	A bandeira começa a dizer "ainda não escrevi nada".
22	O ciclo de fora repete o par saltar/escrever até ao fim da string.
24-25	Salta todos os separadores seguidos.
26	Escreve o espaço separador antes da palavra, e só se já tiver havido outra antes.
28-33	Escreve a palavra caractere a caractere.
32	Assim que sai um caractere, a bandeira passa a dizer "já escrevi". Está dentro do ciclo de propósito: uma string só com espaços nunca chega aqui e a bandeira nunca muda.

N3 expand_str

O mesmo, mas com três espaços entre palavras.

O QUE O ENUNCIADO PEDE

Exactamente o mesmo que o `epur_str`, mas com três espaços entre as palavras em vez de um.

Funções permitidas: `write`.

```
$> ./expand_str " seulement          la c'est          plus dur " | cat -e
seulement  la   c'est  plus   dur$
```

COMO PENSAR

- 1 Muda uma linha. Tudo o resto é idêntico ao `epur_str`.

O CÓDIGO

expand_str.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  int     is_space(char c)
9  {
10     return (c == ' ' || c == '\t');
11 }
12
13 int     main(int argc, char **argv)
14 {
15     int i;
16     int first;
17
18     if (argc == 2)
19     {
20         i = 0;
21         first = 1;
22         while (argv[1][i])
23         {
24             while (is_space(argv[1][i]))
25                 i++;
26             if (argv[1][i] && !first)
27                 write(1, "   ", 3);
28             while (argv[1][i] && !is_space(argv[1][i]))
29             {
30                 ft_putchar(argv[1][i]);
31                 i++;
32                 first = 0;
33             }
34         }
35     }
36     ft_putchar('\n');
37     return (0);
38 }
```

LINHA A LINHA

- 27 Era `ft_putchar(' ')`; passa a escrever três espaços de uma vez. O `3` é quantos bytes se escrevem.

N3 str_capitalizer

A primeira letra de cada palavra em maiúscula, o resto em minúscula.

O QUE O ENUNCIADO PEDE

Para **cada argumento**, pôr a primeira letra de cada palavra em maiúscula e todo o resto em minúscula, e mostrar o resultado seguido de um newline. Uma palavra com uma só letra fica em maiúscula.

Se não houver argumentos, mostrar só um newline.

Funções permitidas: `write`.

```
$> ./str_capitalizer "Premier PETIT Test" | cat -e
Premier Petit Test$
```

COMO PENSAR

- 1 Duas ideias novas. A primeira: este exercício aceita **vários** argumentos, por isso há um ciclo por fora que passa por todos.
- 2 A segunda: como saber se se está no início de uma palavra? Guardando numa bandeira a resposta à pergunta "o caractere anterior era um separador?". A bandeira começa a 1 porque o primeiro caractere da string é sempre início de palavra.
- 3 O truque de organização: **pôr tudo em minúscula primeiro e só depois subir a inicial**. Fazer as duas coisas por ordem é muito mais simples do que tentar decidir tudo de uma vez.

O CÓDIGO

str_capitalizer.c

```
1 #include <unistd.h>
2
3 void    ft_putchar(char c)
4 {
5     write(1, &c, 1);
6 }
7
8 int     is_space(char c)
9 {
10     return (c == ' ' || c == '\t');
11 }
12
13 int     main(int argc, char **argv)
14 {
15     int     a;
16     int     i;
17     int     start;
18     char     c;
19
20     a = 1;
21     while (a < argc)
22     {
23         i = 0;
24         start = 1;
25         while (argv[a][i])
26         {
27             c = argv[a][i];
28             if (c >= 'A' && c <= 'Z')
29                 c = c + 32;
30             if (start && c >= 'a' && c <= 'z')
31                 c = c - 32;
32             start = is_space(argv[a][i]);
33             ft_putchar(c);
34             i++;
35         }
36         ft_putchar('\n');
37         a++;
38     }
39     if (argc == 1)
40         ft_putchar('\n');
41     return (0);
42 }
```

LINHA A LINHA

21	O ciclo de fora: <code>a</code> vai de 1 até ao último argumento.
24	A bandeira começa a dizer "estou no início de uma palavra".
28-29	Passo 1: baixar tudo para minúscula.
30-31	Passo 2: se for início de palavra e for letra, sobe para maiúscula.
32	Actualiza a bandeira para o caractere seguinte : ela passa a valer "o que acabei de ver era separador?". Repara que se olha para <code>argv[a][i]</code> , o caractere original, e não para o <code>c</code> , que já foi modificado.
36	Um newline por argumento.
39-40	O caso de não haver argumentos: o ciclo de fora nunca corre, por isso o newline tem de ser escrito aqui.

N3 rstr_capitalizer

A última letra de cada palavra em maiúscula, o resto em minúscula.

O QUE O ENUNCIADO PEDE

O contrário do `str_capitalizer`: para cada argumento, pôr a **última** letra de cada palavra em maiúscula e o resto em minúscula. Uma palavra com uma só letra fica em maiúscula.

Se não houver argumentos, mostrar só um newline.

Funções permitidas: `write`.

```
$> ./rstr_capitalizer "Premier PETIT Test" | cat -e
premier petit test$
```

COMO PENSAR

- 1 Aqui não é preciso bandeira nenhuma. "Sou o fim de uma palavra?" é uma pergunta sobre o caractere **seguinte**, e esse já se pode ver: é o `argv[a][i + 1]`.
- 2 É seguro espreitar para a frente: se o caractere `i` existe, então o `i + 1` no pior dos casos é o `'\0'`, que também está lá.
- 3 Fim de palavra quer dizer: o seguinte é o `'\0'` ou é um separador.

O CÓDIGO

rstr_capitalizer.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  int     is_space(char c)
9  {
10     return (c == ' ' || c == '\t');
11 }
12
13 int     main(int argc, char **argv)
14 {
15     int    a;
16     int    i;
17     char    c;
18
19     a = 1;
20     while (a < argc)
21     {
22         i = 0;
23         while (argv[a][i])
24         {
25             c = argv[a][i];
26             if (c >= 'A' && c <= 'Z')
27                 c = c + 32;
28             if (c >= 'a' && c <= 'z'
29                 && (argv[a][i + 1] == '\0' || is_space(argv[a][i + 1])))
30                 c = c - 32;
31             ft_putchar(c);
32             i++;
33         }
34         ft_putchar('\n');
35         a++;
36     }
37     if (argc == 1)
38         ft_putchar('\n');
39     return (0);
40 }
```

LINHA A LINHA

26-27 Passo 1, igual ao exercício anterior: baixar tudo.

28-29 Passo 2: é letra **e** o caractere a seguir é o fim da string ou um separador.

30 Só então sobe para maiúscula.

N3 hiddenp

Dizer se a primeira string está escondida na segunda.

O QUE O ENUNCIADO PEDE

Mostrar `1` seguido de newline se a primeira string estiver escondida na segunda, senão `0` seguido de newline.

`s1` está escondida em `s2` se for possível encontrar todos os caracteres de `s1` dentro de `s2`, pela mesma ordem. A string vazia está escondida em qualquer string.

Se o número de parâmetros não for 2, mostrar só um newline.

Funções permitidas: `write`.

```
$> ./hidenp "abc" "2altrb53c.sse" | cat -e
1$
$> ./hidenp "abc" "btarc" | cat -e
0$
```

COMO PENSAR

- 1 É exactamente o mesmo algoritmo do `wdmatch`, no nível 2. A única diferença está no que se escreve no fim.
- 2 Um índice em cada string. O da segunda anda sempre; o da primeira só avança quando encontra o caractere que procurava.
- 3 Se no fim o índice da primeira chegou ao `'\0'`, encontrou tudo pela ordem certa.

O CÓDIGO

hidenp.c

```
1 #include <unistd.h>
2
3 int main(int argc, char **argv)
4 {
5     int i;
6     int j;
7
8     if (argc != 3)
9     {
10         write(1, "\n", 1);
11         return (0);
12     }
13     i = 0;
14     j = 0;
15     while (argv[1][i] && argv[2][j])
16     {
17         if (argv[1][i] == argv[2][j])
18             i++;
19             j++;
20     }
21     if (!argv[1][i])
22         write(1, "1\n", 2);
23     else
24         write(1, "0\n", 2);
25     return (0);
26 }
```

LINHA A LINHA

- | | |
|-------|--|
| 15 | Anda enquanto houver caracteres nas duas strings. |
| 17-18 | O índice da primeira só avança quando bate certo. |
| 19 | O da segunda avança sempre. |
| 21 | <code>!argv[1][i]</code> é "cheguei ao fim da primeira string", ou seja, encontrei tudo. |

Se resolves o `wdmatch`, faz este a seguir — são o mesmo exercício em dois níveis diferentes, e ficam os dois aprendidos de uma vez.

O QUE O ENUNCIADO PEDE

Escrever a função `int *ft_range(int start, int end);`

Tem de alocar com `malloc` um array de inteiros, enchê-lo com os valores consecutivos de `start` até `end` (incluindo os dois) e devolver o ponteiro para o primeiro valor.

Com `(1, 3)` devolve 1, 2, 3. Com `(0, -3)` devolve 0, -1, -2, -3.

Funções permitidas: `malloc`.

COMO PENSAR

- 1 Os três passos do `malloc` outra vez: contar, pedir, encher.
- 2 A dificuldade é só uma: **o array pode ser crescente ou decrescente**, e o enunciado dá exemplos dos dois.
- 3 Resolve-se com um passo (`step`) que vale 1 ou -1, decidido **uma vez** no princípio. Depois as contas funcionam nos dois casos sem repetir código.
- 4 `(end - start) * step` é sempre positivo, seja qual for a direcção, porque o `step` corrige o sinal. Mais um, porque os dois extremos contam.

O CÓDIGO`ft_range.c`

```
1 #include <stdlib.h>
2
3 int *ft_range(int start, int end)
4 {
5     int *tab;
6     int len;
7     int step;
8     int i;
9
10    step = 1;
11    if (start > end)
12        step = -1;
13    len = (end - start) * step + 1;
14    tab = malloc(sizeof(int) * len);
15    if (!tab)
16        return (0);
17    i = 0;
18    while (i < len)
19    {
20        tab[i] = start + i * step;
21        i++;
22    }
23    return (tab);
24 }
```

LINHA A LINHA

10-12 Decide a direcção uma vez só.

13 Quantos elementos. A multiplicação pelo `step` torna a diferença positiva nos dois casos.

14 Pede o espaço. `sizeof(int)` porque são inteiros, não caracteres.

15-16 Verificar o `malloc` é obrigatório. Sem isto, perdem-se pontos mesmo que o programa pareça funcionar.

20 O valor de cada posição: parte do `start` e anda `i` passos. Com `step = -1`, anda para trás sozinho.

Sem a ideia do `step`, este exercício obrigava a escrever tudo a dobrar dentro de um `if`. É o mesmo tipo de simplificação que o `'a' + i` do `maff_alpha`.

N3 `ft_rrange`

O mesmo array, pela ordem inversa.

O QUE O ENUNCIADO PEDE

Exactamente o mesmo que o `ft_range`, mas com os valores pela ordem contrária: começa em `end` e acaba em `start`.

Com `(1, 3)` devolve 3, 2, 1. Com `(0, -3)` devolve -3, -2, -1, 0.

Funções permitidas: `malloc`.

COMO PENSAR

1 Muda uma linha: em vez de partir do `start` e avançar, parte-se do `end` e recua-se.

O CÓDIGO

`ft_rrange.c`

```
1 #include <stdlib.h>
2
3 int *ft_rrange(int start, int end)
4 {
5     int *tab;
6     int len;
7     int step;
8     int i;
9
10    step = 1;
11    if (start > end)
12        step = -1;
13    len = (end - start) * step + 1;
14    tab = malloc(sizeof(int) * len);
15    if (!tab)
16        return (0);
17    i = 0;
18    while (i < len)
19    {
20        tab[i] = end - i * step;
21        i++;
22    }
23    return (tab);
24 }
```

LINHA A LINHA

20 Era `start + i * step`; passa a `end - i * step`. Confere à mão com `(0, -3)`: o `step` é -1, e `-3 - i * (-1)` dá -3, -2, -1, 0. Certo.

N3 `ft_list_size`

Função que conta os elementos de uma lista ligada.

O QUE O ENUNCIADO PEDE

Escrever uma função que devolve o número de elementos da lista que recebe.

A declaração é `int ft_list_size(t_list *begin_list);`

Entregam-se dois ficheiros, e o `ft_list.h` tem de ter exactamente a estrutura do enunciado.

COMO PENSAR

- 1 Uma lista ligada é uma corrente: cada elo guarda um dado e o endereço do elo seguinte. O último aponta para `NULL`, que é o sinal de fim — o equivalente ao `'\0'` das strings.
- 2 Por isso o ciclo é o irmão do ciclo das strings: `while (lista)` em vez de `while (s[i])`.
- 3 Andar para o elo seguinte é `lista = lista->next`.
- 4 Pode-se mexer no ponteiro recebido à vontade: é uma cópia do endereço, e a lista de quem chamou fica intacta.

O CÓDIGO

ft_list_size.c

```
1 #include "ft_list.h"
2
3 int ft_list_size(t_list *begin_list)
4 {
5     int n;
6
7     n = 0;
8     while (begin_list)
9     {
10         n++;
11         begin_list = begin_list->next;
12     }
13     return (n);
14 }
```

ft_list.h

```
1 #ifndef FT_LIST_H
2 # define FT_LIST_H
3
4 typedef struct s_list
5 {
6     struct s_list *next;
7     void *data;
8 } t_list;
9
10 #endif
```

LINHA A LINHA

- 8 `while (begin_list)` lê-se "enquanto o ponteiro não for `NULL`". O `NULL` vale zero, logo conta como falso, tal como o `'\0'`.
-
- 10 Mais um elo contado.
-
- 11 `->` é a forma de chegar a um campo através de um ponteiro. `begin_list->next` é o endereço do elo seguinte.

O `#ifndef` do cabeçalho é a protecção normal dos `.h`: evita que o ficheiro seja incluído duas vezes. Não é obrigatório, mas nunca prejudica. O `typedef` tem de ser copiado **do enunciado**, não escrito de memória.

N4 Nível 4

11 exercícios. Memória dentro de memória, ordenação e ponteiros para funções. É aqui que a maior parte das pessoas encaixa.

N4 sort_int_tab

Função que ordena um array de inteiros no próprio sítio.

O QUE O ENUNCIADO PEDE

Escrever a função `void sort_int_tab(int *tab, unsigned int size);`

Ordena o array por ordem crescente, no próprio sítio. Os valores repetidos mantêm-se todos.

COMO PENSAR

- 1 É o mais fácil do nível 4 — começa por este.
- 2 A ideia mais simples que funciona: percorrer o array comparando cada elemento com o seguinte. Se estiverem trocados, trocam-se. Depois volta-se ao princípio. Quando se chegar ao fim sem trocar nada, está ordenado.
- 3 Voltar ao princípio a cada troca parece lento, e é, mas o exame não mede velocidade — mede se o output está certo.
- 4 A troca é a mesma do `ft_swap`: precisa de uma variável temporária.

O CÓDIGO

sort_int_tab.c

```
1 void sort_int_tab(int *tab, unsigned int size)
2 {
3     unsigned int i;
4     int tmp;
5
6     i = 0;
7     while (i + 1 < size)
8     {
9         if (tab[i] > tab[i + 1])
10        {
11            tmp = tab[i];
12            tab[i] = tab[i + 1];
13            tab[i + 1] = tmp;
14            i = 0;
15        }
16        else
17            i++;
18    }
19 }
```

LINHA A LINHA

- | | |
|-------|--|
| 3 | O <code>i</code> tem de ser <code>unsigned int</code> como o <code>size</code> . Comparar tipos com sinais diferentes dá aviso, e aviso é exercício falhado. |
| 7 | <code>i + 1 < size</code> garante que o <code>tab[i + 1]</code> da linha seguinte existe sempre. |
| 9 | Estão fora de ordem? |
| 11-13 | A troca em três passos, com o <code>tmp</code> a guardar um dos valores. |
| 14 | Volta ao princípio. É esta linha que faz o algoritmo funcionar: uma troca pode ter desarrumado algo mais atrás. |
| 16-17 | Se estavam pela ordem certa, avança. |

O QUE O ENUNCIADO PEDE

Receber um inteiro positivo e mostrar os seus factores primos por ordem crescente, separados por `*`, seguidos de um newline.

Se o número de parâmetros não for 1, mostrar só o newline. O número de entrada, quando existe, é sempre válido.

Funções permitidas: `printf`, `atoi`.

```
$> ./fprime 225225 | cat -e
3*3*5*5*7*11*13$
$> ./fprime 42 | cat -e
2*3*7$
$> ./fprime 1 | cat -e
1$
```

COMO PENSAR

- ❶ Não é preciso saber quais são os primos. Basta dividir por 2 enquanto der, depois por 3 enquanto der, depois por 4... Quando se chega ao 4, o número já não tem factores 2 nenhuns, por isso a divisão nunca dá certo. Os divisores que funcionam são automaticamente primos.
- ❷ O `1` é um caso à parte: não tem factores, e o enunciado diz que o output é `1`.
- ❸ O `*` escreve-se **antes** de cada factor menos o primeiro — a mesma ideia do separador do `epur_str`.

O CÓDIGO

fprime.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int  main(int argc, char **argv)
5  {
6      int n;
7      int d;
8      int first;
9
10     if (argc != 2)
11     {
12         printf("\n");
13         return (0);
14     }
15     n = atoi(argv[1]);
16     if (n <= 1)
17     {
18         printf("%d\n", n);
19         return (0);
20     }
21     first = 1;
22     d = 2;
23     while (n > 1)
24     {
25         while (n % d == 0)
26         {
27             if (!first)
28                 printf("*");
29             printf("%d", d);
30             first = 0;
31             n = n / d;
32         }
33         d++;
34     }
35     printf("\n");
36     return (0);
37 }
```

LINHA A LINHA

16-20 O caso do 1 (e do 0), tratado à parte.

21 A bandeira do separador.

23 Enquanto sobrar alguma coisa para decompor.

25 Enquanto o divisor actual ainda couber, aplica-se repetidamente — é assim que o 225225 dá dois 3 e dois 5.

27-28 O `*` antes de cada factor excepto o primeiro.

30 A partir daqui já houve um factor, logo os seguintes levam `*`.

31 Retira-se o factor do número.

33 Só depois de o divisor já não caber é que se passa ao seguinte.

N4 rostring

Passar a primeira palavra para o fim da frase.

O QUE O ENUNCIADO PEDE

Mostrar a string depois de a rodar uma palavra para a esquerda: a primeira palavra passa a última e as outras mantêm a ordem. As palavras saem separadas por um só espaço.

Se houver menos de um argumento, mostrar só um newline.

Funções permitidas: `write`, `malloc`, `free`.

```
$> ./rostring "Que la      lumiere soit et la lumiere fut"
la lumiere soit et la lumiere fut Que
$> ./rostring "abc  " | cat -e
abc$
```

COMO PENSAR

- ❶ Não é preciso `malloc` nenhum, apesar de ser permitido: dá para escrever tudo directamente na ordem certa.
- ❷ O plano: guardar onde começa e acaba a primeira palavra, escrever todas as outras com o espaçamento do `epur_str`, e no fim escrever a primeira.
- ❸ O segundo exemplo é o caso que apanha muita gente: se só houver uma palavra, não pode sair espaço nenhum antes dela.

O CÓDIGO

rostring.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  int     is_space(char c)
9  {
10     return (c == ' ' || c == '\t');
11 }
12
13 int     main(int argc, char **argv)
14 {
15     int i;
16     int start;
17     int end;
18     int written;
19
20     if (argc < 2)
21     {
22         ft_putchar('\n');
23         return (0);
24     }
25     i = 0;
26     while (is_space(argv[1][i]))
27         i++;
28     start = i;
29     while (argv[1][i] && !is_space(argv[1][i]))
30         i++;
31     end = i;
32     written = 0;
33     while (argv[1][i])
34     {
35         while (is_space(argv[1][i]))
36             i++;
37         if (argv[1][i] && written)
38             ft_putchar(' ');
39         while (argv[1][i] && !is_space(argv[1][i]))
40         {
41             ft_putchar(argv[1][i]);
42             written = 1;
43             i++;
44         }
45     }
46     if (written && end > start)
47         ft_putchar(' ');
48     while (start < end)
49         ft_putchar(argv[1][start++]);
50     ft_putchar('\n');
51     return (0);
52 }
```

LINHA A LINHA

- 25–31 Localiza a primeira palavra: salta os separadores do início, marca o `start`, passa por cima da palavra, marca o `end`. Não escreve nada ainda.
- 32 A bandeira "já escrevi alguma palavra", igual à do `epur_str`.
- 33–45 Escreve todas as palavras a partir da segunda, com um espaço antes de cada uma menos da primeira que for escrita.
- 46–47 O espaço antes da primeira palavra, que agora vai para o fim. Só se escreve se já tiver saído alguma coisa e se a primeira palavra existir mesmo.
- 48–49 Escreve a primeira palavra, do `start` ao `end`. O `start++` dentro dos parênteses rectos avança o índice ao mesmo tempo que o usa.

N4 rev_wstr

Escrever as palavras pela ordem inversa.

O QUE O ENUNCIADO PEDE

Mostrar as palavras da string pela ordem inversa, seguidas de um newline. Inverte-se a ordem das **palavras**, não as letras de cada uma.

Se o número de parâmetros não for 1, mostrar só o newline.

Funções permitidas: `write`, `malloc`, `free`.

```
$> ./rev_wstr "il contempla le mont" | cat -e
mont le contempla il$
```

COMO PENSAR

- 1 É o `last_word` posto dentro de um ciclo: encontrar a última palavra, escrevê-la, e continuar a recuar para a anterior.
- 2 Também não precisa de `malloc`: percorre-se a string de trás para a frente e escreve-se à medida.
- 3 Cada volta: saltar os separadores a recuar, marcar o fim da palavra, recuar por cima da palavra, e escrever do princípio dela até ao fim.
- 4 Repara que depois de recuar por cima da palavra o índice fica **antes** dela, e é exactamente onde o ciclo seguinte tem de continuar — não é preciso corrigir nada.

O CÓDIGO

rev_wstr.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  int     is_space(char c)
9  {
10     return (c == ' ' || c == '\t');
11 }
12
13 int     main(int argc, char **argv)
14 {
15     int i;
16     int j;
17     int end;
18     int written;
19
20     if (argc != 2)
21     {
22         ft_putchar('\n');
23         return (0);
24     }
25     i = 0;
26     while (argv[1][i])
27         i++;
28     i--;
29     written = 0;
30     while (i >= 0)
31     {
32         while (i >= 0 && is_space(argv[1][i]))
33             i--;
34         end = i;
35         while (i >= 0 && !is_space(argv[1][i]))
36             i--;
37         if (end > i)
38         {
39             if (written)
40                 ft_putchar(' ');
41             written = 1;
42             j = i + 1;
43             while (j <= end)
44                 ft_putchar(argv[1][j++]);
45         }
46     }
47     ft_putchar('\n');
48     return (0);
49 }
```

LINHA A LINHA

26–28 Vai ao fim da string e recua um, para o último caractere verdadeiro.

30 O ciclo de fora anda para trás até ao princípio da string.

32–33 Salta os separadores, a recuar.

34 Marca o fim da palavra.

35–36 Recua por cima da palavra toda.

37 `end > i` quer dizer "encontrei mesmo uma palavra". Se a string acabar em separadores, esta condição evita escrever uma palavra vazia.

39–40 O espaço antes de cada palavra menos da primeira a sair.

42–44 Escreve a palavra para a frente: começa em `i + 1`, que é o primeiro caractere dela, e vai até ao `end`.

N4 `ft_split`

Função que parte uma string num array de palavras.

O QUE O ENUNCIADO PEDE

Escrever uma função que parte uma string em palavras e as devolve num array de strings terminado por `NULL`.

Uma palavra é um pedaço de string delimitado por espaços, tabs ou newlines, ou pelo princípio e fim da string.

A declaração é `char **ft_split(char *str);`

Funções permitidas: `malloc`.

COMO PENSAR

- 1 Aqui há `malloc` dentro de `malloc`: um para a **lista de ponteiros** e um por cada palavra.
- 2 Por isso é preciso saber quantas palavras há **antes** de pedir a lista — daí a função que só conta.
- 3 O `NULL` no fim é o que permite a quem chama saber onde a lista acaba, tal como o `'\0'` nas strings. Esquecê-lo é o erro clássico deste exercício.
- 4 Dividir o trabalho em três funções pequenas (contar, copiar uma palavra, montar) torna isto muito mais fácil do que tentar fazer tudo numa.

O CÓDIGO

`ft_split.c`

```

1  #include <stdlib.h>
2
3  int    is_sep(char c)
4  {
5      return (c == ' ' || c == '\t' || c == '\n');
6  }
7
8  int    count_words(char *str)
9  {
10     int i;
11     int n;
12
13     i = 0;
14     n = 0;
15     while (str[i])
16     {
17         while (str[i] && is_sep(str[i]))
18             i++;
19         if (str[i])
20             n++;
21         while (str[i] && !is_sep(str[i]))
22             i++;
23     }
24     return (n);
25 }
26
27 char    *word_dup(char *str, int start, int end)
28 {
29     char    *w;
30     int      i;
31
32     w = malloc(sizeof(char) * (end - start + 1));
33     if (!w)
34         return (0);
35     i = 0;
36     while (start < end)
37         w[i++] = str[start++];
38     w[i] = '\0';
39     return (w);
40 }
41
42 char    **ft_split(char *str)
43 {
44     char    **tab;
45     int      i;
46     int      j;
47     int      start;
48
49     tab = malloc(sizeof(char *) * (count_words(str) + 1));
50     if (!tab)
51         return (0);
52     i = 0;
53     j = 0;
54     while (str[i])
55     {
56         while (str[i] && is_sep(str[i]))
57             i++;
58         start = i;
59         while (str[i] && !is_sep(str[i]))
60             i++;
61         if (i > start)
62             tab[j++] = word_dup(str, start, i);
63     }
64     tab[j] = 0;
65     return (tab);
66 }

```

LINHA A LINHA

3-6	Os separadores. Aqui o enunciado inclui o <code>\n</code> , ao contrário dos exercícios de palavras dos níveis anteriores.
8-25	Só conta palavras. É o mesmo par de ciclos de sempre, mas em vez de escrever, soma um.
19-20	Conta a palavra no momento em que a encontra, antes de passar por cima dela.
27-40	Copia uma palavra para memória nova. Recebe onde ela começa e onde acaba.
32	<code>end - start</code> é o comprimento da palavra; o <code>+ 1</code> é o espaço do <code>'\0'</code> .
38	O <code>'\0'</code> posto à mão, como sempre depois de um <code>malloc</code> .
49	A lista de ponteiros: tantos quantas as palavras, mais um para o <code>NULL</code> final. <code>sizeof(char *)</code> porque são endereços de strings , não caracteres.
61-62	Só guarda a palavra se ela existir mesmo. O <code>j++</code> avança a posição na lista.
64	O <code>NULL</code> final. Sem ele, quem percorre a lista não sabe onde parar.

N4 ft_itoa

Função que converte um inteiro numa string nova.

O QUE O ENUNCIADO PEDE

Escrever uma função que recebe um `int` e o converte numa string terminada em zero. A string é devolvida em memória que a função tem de alocar.

A declaração é `char *ft_itoa(int nbr);`

Funções permitidas: `malloc`.

COMO PENSAR

- 1 É o contrário do `ft_atoi`, e também é a `ft_putnbr` mas a guardar em vez de escrever.
- 2 Como é preciso `malloc`, tem de se saber o tamanho **antes**: conta-se quantos algarismos tem o número dividindo por 10 até sobrar um só.
- 3 Se for negativo, é mais um caractere para o sinal.
- 4 Preenche-se **de trás para a frente**, porque o resto da divisão por 10 dá sempre o **último** algarismo. Assim não é preciso inverter nada no fim.
- 5 O `n` é declarado `unsigned int` e não `int` por causa do `-2147483648`: esse número não tem positivo dentro de um `int`, e `-n` daria o próprio número outra vez. Num `unsigned int` o 2147483648 cabe à vontade.

O CÓDIGO

ft_itoa.c

```
1  #include <stdlib.h>
2
3  char  *ft_itoa(int nbr)
4  {
5      char      *s;
6      unsigned int  n;
7      int      len;
8      int      neg;
9
10     neg = 0;
11     n = nbr;
12     if (nbr < 0)
13     {
14         neg = 1;
15         n = -(unsigned int)nbr;
16     }
17     len = 1;
18     while (n >= 10)
19     {
20         n = n / 10;
21         len++;
22     }
23     len = len + neg;
24     s = malloc(sizeof(char) * (len + 1));
25     if (!s)
26         return (0);
27     s[len] = '\0';
28     n = nbr;
29     if (neg)
30         n = -(unsigned int)nbr;
31     while (len-- > neg)
32     {
33         s[len] = n % 10 + '0';
34         n = n / 10;
35     }
36     if (neg)
37         s[0] = '-';
38     return (s);
39 }
```

LINHA A LINHA

- 6 `unsigned int` em vez de `int` — é o que permite guardar o simétrico do menor `int` possível, o 2147483648 que num `int` não cabe. Um `long` também servia em Linux, mas em Windows o `long` tem 32 bits e voltava a não caber; o `unsigned int` funciona nas duas.
- 12-16 Se é negativo, marca-o e trabalha com o valor positivo. O `-(unsigned int)nbr` converte **primeiro** e só depois troca o sinal: em `unsigned` essa conta é circular e definida pela norma, ao contrário do `-nbr` num `int`, que para o `-2147483648` é transbordo.
- 17-22 Conta os algarismos: divide por 10 até sobrar um só. O `len` começa em 1 porque mesmo o zero tem um algarismo.
- 23 Se é negativo, mais um caractere para o `-`.
- 24 Pede o espaço, com o `+ 1` do `'\0'`.
- 27 O `'\0'` no fim, escrito já.
- 28-30 Repõe o número, porque o ciclo de contagem destruiu-o.
- 31 `len-- > neg` faz duas coisas: compara e depois recua. Pára em `neg`, deixando a posição o livre para o sinal quando é preciso.
- 33 `n % 10` é o último algarismo; somar `'0'` transforma-o em caractere.
- 36-37 O sinal, na primeira posição.

N4 ft_list_foreach

Função que aplica uma função a cada elemento de uma lista.

O QUE O ENUNCIADO PEDE

Escrever uma função que recebe uma lista e um ponteiro para função, e aplica essa função a cada elemento da lista.

A declaração é `void ft_list_foreach(t_list *begin_list, void (*f)(void *));`

A função apontada por `f` é usada assim: `(*f)(list_ptr->data);`

Entregam-se dois ficheiros: o `.c` e o `ft_list.h` com a estrutura do enunciado.

COMO PENSAR

- 1 É o ciclo do `ft_list_size` com uma linha diferente lá dentro.
- 2 A novidade é o **ponteiro para função**: `f` não é um valor, é o endereço de uma função que alguém escreveu noutro sítio. Chamá-la escreve-se `(*f)(argumento)`.
- 3 Não é preciso perceber o que a função faz — só chamá-la com o dado de cada elo.

O CÓDIGO

`ft_list_foreach.c`

```
1 #include "ft_list.h"
2
3 void ft_list_foreach(t_list *begin_list, void (*f)(void *))
4 {
5     while (begin_list)
6     {
7         (*f)(begin_list->data);
8         begin_list = begin_list->next;
9     }
10 }
```

```

1  #ifndef FT_LIST_H
2  # define FT_LIST_H
3
4  typedef struct s_list
5  {
6      struct s_list  *next;
7      void            *data;
8  } t_list;
9
10 #endif

```

LINHA A LINHA

- 3 `void (*f)(void *)` lê-se de dentro para fora: `f` é um ponteiro para uma função que recebe um `void *` e não devolve nada.
- 5 O ciclo das listas: anda enquanto o ponteiro não for `NULL`.
- 7 Chama a função com o dado deste elo, exactamente como o enunciado manda escrever.
- 8 Passa ao elo seguinte.

N4 ft_list_remove_if

Função que apaga da lista os elementos iguais a um dado de referência.

O QUE O ENUNCIADO PEDE

Escrever uma função que remove da lista todos os elementos cujo dado seja "igual" ao dado de referência.

A declaração é `void ft_list_remove_if(t_list **begin_list, void *data_ref, int (*cmp)());`

O `cmp` recebe dois `void *` e devolve o quando os dois são iguais.

Funções permitidas: `free`.

COMO PENSAR

- ❶ Reparar em duas coisas no enunciado. O `cmp` devolve **o quando são iguais** — ao contrário do que a intuição diz. E o primeiro parâmetro é `t_list **`, ou seja, o endereço **do ponteiro** para o início: é assim que a função consegue mudar a cabeça da lista.
- ❷ Apagar um elo do meio é fácil: o elo anterior passa a apontar para o seguinte, e liberta-se o do meio.
- ❸ Apagar a **cabeça** é diferente, porque não há elo anterior. Por isso trata-se primeiro, num ciclo à parte, até a cabeça já não ser para apagar.
- ❹ Depois de apagar um elo do meio **não se avança**: o elo seguinte também pode ser para apagar.

O CÓDIGO

ft_list_remove_if.c

```
1 #include <stdlib.h>
2 #include "ft_list.h"
3
4 void ft_list_remove_if(t_list **begin_list, void *data_ref, int (*cmp)())
5 {
6     t_list *cur;
7     t_list *tmp;
8
9     while (*begin_list && cmp((*begin_list)->data, data_ref) == 0)
10    {
11        tmp = (*begin_list)->next;
12        free(*begin_list);
13        *begin_list = tmp;
14    }
15    cur = *begin_list;
16    while (cur && cur->next)
17    {
18        if (cmp(cur->next->data, data_ref) == 0)
19        {
20            tmp = cur->next->next;
21            free(cur->next);
22            cur->next = tmp;
23        }
24        else
25            cur = cur->next;
26    }
27 }
```

LINHA A LINHA

- | | |
|-------|---|
| 9 | Primeiro ciclo: enquanto a cabeça for para apagar. <code>*begin_list</code> é o ponteiro para o primeiro elo. |
| 11-13 | Guarda o seguinte, liberta a cabeça, e a cabeça passa a ser esse seguinte. É aqui que a lista de quem chamou muda mesmo. |
| 15 | A partir daqui a cabeça já está garantidamente certa. |
| 16 | Segundo ciclo: olha sempre para o elo a seguir ao actual, porque é preciso ter o anterior à mão para religar a corrente. |
| 20-22 | Guarda o elo a seguir ao que vai desaparecer, liberta-o, e liga o actual directamente a esse. A corrente fica fechada. |
| 24-25 | Só se avança quando o elo seguinte não foi apagado. |

N4 sort_list

Função que ordena uma lista ligada com um comparador dado.

O QUE O ENUNCIADO PEDE

Escrever a função `t_list *sort_list(t_list* lst, int (*cmp)(int, int));`

Ordena a lista usando o `cmp` para decidir a ordem, e devolve o ponteiro para o primeiro elemento. Os valores repetidos mantêm-se.

O `cmp` devolve um valor diferente de 0 quando `a` e `b` **estão na ordem certa**. Por exemplo, esta função usada como `cmp` ordena por ordem crescente:

```
int ascending(int a, int b) { return (a <= b); }
```

O tipo `t_list` vem do ficheiro `list.h` que é fornecido. **Inclui-se esse ficheiro mas não se entrega.**

COMO PENSAR

- 1 É o `sort_int_tab` aplicado a uma lista.

- ❷ O truque que simplifica tudo: **trocar os dados em vez dos elos**. Reorganizar ponteiros é muito mais complicado e o resultado visto de fora é exactamente o mesmo.
- ❸ Quando há uma troca, volta-se ao princípio da lista, tal como no `sort_int_tab`.
- ❹ Atenção ao sentido do `cmp`: devolve verdadeiro quando está **bem**, por isso troca-se quando ele devolve falso — daí o `!`.

O CÓDIGO

`sort_list.c`

```
1  #include "list.h"
2
3  t_list *sort_list(t_list *lst, int (*cmp)(int, int))
4  {
5      t_list *cur;
6      int     tmp;
7
8      cur = lst;
9      while (cur && cur->next)
10     {
11         if (!(*cmp)(cur->data, cur->next->data))
12         {
13             tmp = cur->data;
14             cur->data = cur->next->data;
15             cur->next->data = tmp;
16             cur = lst;
17         }
18         else
19             cur = cur->next;
20     }
21     return (lst);
22 }
```

LINHA A LINHA

9 Anda enquanto houver um elo actual e um seguinte para comparar. Também trata o caso da lista vazia.

11 `!(*cmp)(...)` é "não estão na ordem certa". O `!` é fácil de esquecer e inverte o resultado todo.

13-15 Troca os **dados** dos dois elos. Os elos ficam onde estão.

16 Volta ao princípio, porque a troca pode ter desarrumado algo mais atrás.

18-19 Se estavam bem, avança para o elo seguinte.

21 Devolve a mesma cabeça que recebeu: como só se trocaram dados, ela nunca muda.

N4 flood_fill

Função que pinta uma zona ligada de um mapa a duas dimensões.

O QUE O ENUNCIADO PEDE

Escrever uma função que recebe um `char **` (um mapa a duas dimensões), um `t_point` com as dimensões e um `t_point` com o ponto de partida.

A partir desse ponto, preenche toda a zona substituindo os caracteres por `'F'`. Uma zona é um grupo do mesmo caractere delimitado na horizontal e na vertical por outros caracteres ou pelo limite do mapa. **Não preenche na diagonal.**

A declaração é `void flood_fill(char **tab, t_point size, t_point begin);`

COMO PENSAR

- 1 O problema resolve-se sozinho com recursão: pinta-se a casa actual e pede-se ao mesmo algoritmo que pinte as quatro vizinhas.
- 2 Uma função recursiva precisa sempre de casos de paragem, e aqui são dois: sair fora do mapa, e chegar a uma casa que não é da cor que se está a pintar.
- 3 Pintar a casa **antes** de chamar as vizinhas é o que impede o programa de andar em círculos: quando uma vizinha tentar voltar para trás, já não encontra a cor original.
- 4 Nota sobre os índices: `tab[y][x]` — a linha é o `y` e a coluna é o `x`, por esta ordem.

O CÓDIGO

flood_fill.c

```
1 #include "flood_fill.h"
2
3 void fill(char **tab, t_point size, t_point p, char target)
4 {
5     t_point next;
6
7     if (p.x < 0 || p.x >= size.x || p.y < 0 || p.y >= size.y)
8         return ;
9     if (tab[p.y][p.x] != target)
10        return ;
11     tab[p.y][p.x] = 'F';
12     next.y = p.y;
13     next.x = p.x + 1;
14     fill(tab, size, next, target);
15     next.x = p.x - 1;
16     fill(tab, size, next, target);
17     next.x = p.x;
18     next.y = p.y + 1;
19     fill(tab, size, next, target);
20     next.y = p.y - 1;
21     fill(tab, size, next, target);
22 }
23
24 void flood_fill(char **tab, t_point size, t_point begin)
25 {
26     if (tab[begin.y][begin.x] == 'F')
27         return ;
28     fill(tab, size, begin, tab[begin.y][begin.x]);
29 }
```

flood_fill.h

```
1 #ifndef FLOOD_FILL_H
2 # define FLOOD_FILL_H
3
4 typedef struct s_point
5 {
6     int x;
7     int y;
8 } t_point;
9
10 void fill(char **tab, t_point size, t_point p, char target);
11 void flood_fill(char **tab, t_point size, t_point begin);
12
13 #endif
```

LINHA A LINHA

7-8	Primeiro caso de paragem: fora do mapa.
9-10	Segundo caso de paragem: esta casa não é da cor que se está a pintar.
11	Pinta. Tem de ser antes das chamadas seguintes, senão o programa entra em ciclo infinito.
12-21	As quatro vizinhas: direita, esquerda, baixo, cima. A variável <code>next</code> é reaproveitada de cada vez.
26-27	Se o ponto de partida já for <code>'F'</code> , não há nada a fazer — e esta guarda evita a recursão infinita nesse caso.
28	Arranca a recursão, passando como alvo a cor que está no ponto de partida.

N4 check_mate

Dizer se o rei está em xeque num tabuleiro dado.

O QUE O ENUNCIADO PEDE

O programa recebe as linhas de um tabuleiro como argumentos e verifica se o rei está em xeque. Só existem peões (`P`), bispos (`B`), torres (`R`), uma dama (`Q`) e um rei (`K`). Tudo o resto conta como casa vazia.

O tabuleiro é sempre quadrado, mas o tamanho varia. Há só um rei e todas as outras peças são inimigas. Uma peça só captura a **primeira** peça que encontra no seu caminho.

Se o rei puder ser capturado, mostrar `Success` seguido de newline; senão `Fail`. Sem argumentos, mostrar só um newline.

Funções permitidas: `write`, `malloc`, `free`.

```
$> ./check_mate 'R...' '..P.' '.K..' '....' | cat -e
Success$
$> ./check_mate '..' '.K' | cat -e
Fail$
```

COMO PENSAR

- 1 Em vez de perguntar "que peças atacam o rei?", faz-se a pergunta ao contrário: **partindo do rei, o que há em cada direcção?** Assim não é preciso percorrer o tabuleiro todo por cada peça.
- 2 São oito direcções: quatro a direito (onde atacam torres e damas) e quatro na diagonal (bispos e damas). Em cada uma anda-se até encontrar uma peça ou sair do tabuleiro. **A primeira peça encontrada bloqueia a direcção**, ataque ou não.
- 3 O peão é o único caso à parte, porque só captura a uma casa de distância e na diagonal. Neste enunciado, o peão captura para baixo — vê-se pelo segundo exemplo, onde o `P` está na linha acima do rei.
- 4 O tabuleiro não precisa de ser copiado: o `argv + 1` já é um `char **` com as linhas todas.

O CÓDIGO

`check_mate.c`

```

1  #include <unistd.h>
2  #include "check_mate.h"
3
4  int    is_piece(char c)
5  {
6      return (c == 'P' || c == 'B' || c == 'R' || c == 'Q');
7  }
8
9  int    ray(char **b, int n, t_pos k, t_pos d)
10 {
11     int r;
12     int c;
13
14     r = k.r + d.r;
15     c = k.c + d.c;
16     while (r >= 0 && r < n && c >= 0 && c < n)
17     {
18         if (is_piece(b[r][c]))
19         {
20             if (d.r == 0 || d.c == 0)
21                 return (b[r][c] == 'R' || b[r][c] == 'Q');
22             return (b[r][c] == 'B' || b[r][c] == 'Q');
23         }
24         r = r + d.r;
25         c = c + d.c;
26     }
27     return (0);
28 }
29
30 int    in_check(char **b, int n, t_pos k)
31 {
32     int    dr[8] = {-1, 1, 0, 0, -1, -1, 1, 1};
33     int    dc[8] = {0, 0, -1, 1, -1, 1, -1, 1};
34     t_pos  d;
35     int    i;
36
37     i = 0;
38     while (i < 8)
39     {
40         d.r = dr[i];
41         d.c = dc[i];
42         if (ray(b, n, k, d))
43             return (1);
44         i++;
45     }
46     if (k.r > 0 && k.c > 0 && b[k.r - 1][k.c - 1] == 'P')
47         return (1);
48     if (k.r > 0 && k.c + 1 < n && b[k.r - 1][k.c + 1] == 'P')
49         return (1);
50     return (0);
51 }
52
53 int    main(int argc, char **argv)
54 {
55     char    **b;
56     int     n;
57     t_pos   k;
58
59     if (argc < 2)
60     {
61         write(1, "\n", 1);
62         return (0);
63     }
64     b = argv + 1;
65     n = argc - 1;
66     k.r = 0;
67     while (k.r < n)

```

```

68     {
69         k.c = 0;
70         while (b[k.r][k.c])
71         {
72             if (b[k.r][k.c] == 'K' && in_check(b, n, k))
73             {
74                 write(1, "Success\n", 8);
75                 return (0);
76             }
77             k.c++;
78         }
79         k.r++;
80     }
81     write(1, "Fail\n", 5);
82     return (0);
83 }

```

check_mate.h

```

1  #ifndef CHECK_MATE_H
2  # define CHECK_MATE_H
3
4  typedef struct s_pos
5  {
6      int r;
7      int c;
8  } t_pos;
9
10 int is_piece(char c);
11 int ray(char **b, int n, t_pos k, t_pos d);
12 int in_check(char **b, int n, t_pos k);
13
14 #endif

```

LINHA A LINHA

- 4-7 O que conta como peça. Tudo o resto é casa vazia, como o enunciado manda.
- 9 Uma direcção de cada vez. O `d` diz quanto andar em linha e em coluna a cada passo.
- 14-15 Começa na casa a seguir ao rei, não no próprio rei.
- 16 Anda enquanto estiver dentro do tabuleiro.
- 18 Encontrou uma peça: seja qual for, esta direcção acaba aqui.
- 20-21 Se a direcção é a direito (uma das componentes é zero), só torre e dama atacam.
- 22 Senão é diagonal, e só bispo e dama atacam.
- 32-33 As oito direcções, guardadas em dois arrays: as quatro primeiras a direito, as quatro últimas na diagonal.
- 38-45 Testa as oito. Basta uma atacar para o rei estar em xeque.
- 46-49 Os dois peões possíveis: na linha de cima, uma coluna para cada lado. Os testes `k.r > 0` e `k.c + 1 < n` evitam ler fora do tabuleiro.
- 64-65 O tabuleiro são os argumentos a partir do primeiro, e o tamanho é quantos são.
- 67-79 Procura o rei. Assim que o encontra, testa-o.

A direcção de captura do peão é o único ponto ambíguo deste enunciado. Aqui seguiu-se o segundo exemplo do próprio enunciado, que só dá `Success` se o peão capturar para baixo. Se o teu enunciado trouxer um `examples.txt` com um desenho diferente, as linhas a trocar são a 46 e a 48.

N5 Nível 5

6 exercícios. Os mais difíceis da pool. Se sair um destes no exame, já há nota para passar.

N5 brackets

Verificar se os parênteses de cada argumento estão bem fechados.

O QUE O ENUNCIADO PEDE

Para cada argumento, mostrar `OK` seguido de newline se a expressão estiver bem emparelhada, senão `Error` seguido de newline.

Contam como parênteses o `(` e `)`, o `[` e `]`, e o `{` e `}`. Todos os outros caracteres são ignorados. Um parêntese aberto tem de ser fechado pelo parêntese certo e pela ordem certa. Uma string sem parênteses nenhuns está bem emparelhada.

Sem argumentos, mostrar só um newline.

Funções permitidas: `write`.

```
$> ./brackets '(johndoe)' | cat -e
OK$
$> ./brackets '([])' | cat -e
Error$
```

COMO PENSAR

- 1 A estrutura certa para isto chama-se **pilha**: uma lista onde só se mete e tira pelo topo, como uma pilha de pratos.
- 2 A ideia: quando aparece um parêntese aberto, guarda-se no topo qual é o **fecho** que se espera. Quando aparece um fechado, compara-se com o topo e tira-se.
- 3 Se o que aparece não bate certo com o topo, está errado. Se aparecer um fechado com a pilha vazia, também. Se no fim sobrar alguma coisa na pilha, também.
- 4 O terceiro caso é o que distingue `([])` de `([()])`: os dois têm os mesmos caracteres, só muda a ordem, e é o topo da pilha que a detecta.
- 5 Uma pilha faz-se com um array e um índice para o topo. Não é preciso `malloc` nenhum.

O CÓDIGO

brackets.c

```
1  #include <unistd.h>
2  #include "brackets.h"
3
4  int    is_open(char c)
5  {
6      return (c == '(' || c == '[' || c == '{');
7  }
8
9  char    closer(char c)
10 {
11     if (c == '(')
12         return (')');
13     if (c == '[')
14         return (']');
15     return ('}');
16 }
17
18 int    check(char *s)
19 {
20     char    stack[4096];
21     int    top;
22     int    i;
23
24     top = 0;
25     i = 0;
26     while (s[i])
27     {
28         if (is_open(s[i]) && top < 4096)
29             stack[top++] = closer(s[i]);
30         else if (s[i] == ')' || s[i] == ']' || s[i] == '}')
31         {
32             if (top == 0 || stack[--top] != s[i])
33                 return (0);
34         }
35         i++;
36     }
37     return (top == 0);
38 }
39
40 int    main(int argc, char **argv)
41 {
42     int i;
43
44     if (argc < 2)
45     {
46         write(1, "\n", 1);
47         return (0);
48     }
49     i = 1;
50     while (i < argc)
51     {
52         if (check(argv[i]))
53             write(1, "OK\n", 3);
54         else
55             write(1, "Error\n", 6);
56         i++;
57     }
58     return (0);
59 }
```

```

1 #ifndef BRACKETS_H
2 # define BRACKETS_H
3
4 int      is_open(char c);
5 char     closer(char c);
6 int      check(char *s);
7
8 #endif

```

LINHA A LINHA

4-7	Os três caracteres que abrem.
9-16	Dado um que abre, qual é o que fecha. O último <code>return</code> cobre o <code>{</code> porque já não há mais hipóteses.
20-21	A pilha e o índice do topo. O <code>top</code> é sempre "quantos elementos há lá dentro".
28-29	Abriu: guarda no topo o fecho esperado. <code>stack[top++]</code> guarda e depois sobe o topo.
30	Fechou: só se faz alguma coisa se for mesmo um dos três caracteres de fecho.
32	Duas maneiras de falhar numa linha: a pilha está vazia, ou o topo não é o fecho esperado. <code>stack[--top]</code> desce o topo e depois lê — é a operação de tirar da pilha.
37	No fim tem de estar vazia. Se sobrou algo aberto, é erro.
50-56	Um <code>OK</code> ou <code>Error</code> por cada argumento.

N5 ft_itoa_base

Função que converte um inteiro numa string, em qualquer base de 2 a 16.

O QUE O ENUNCIADO PEDE

Escrever uma função que converte um valor inteiro numa string terminada em zero, na base indicada, guardando o resultado em memória que a função tem de alocar.

A base é um inteiro de 2 a 16. Os caracteres são os algarismos de 0 a 9 seguidos das **maiúsculas** de A a F.

Se a base for 10 e o valor for negativo, a string leva um `-` à frente. **Em qualquer outra base o valor é tratado como sem sinal.**

A declaração é `char *ft_itoa_base(int value, int base);`

Funções permitidas: `malloc`.

COMO PENSAR

- 1 É o `ft_itoa` do nível 4 com a base como variável, mais a regra especial do sinal.
- 2 A regra do sinal é o que torna este mais difícil: só em base 10 é que um número negativo leva `-`. Nas outras bases, o mesmo número tem de ser lido como um valor positivo enorme — é por isso que se trabalha com `unsigned int` em vez de `int`.
- 3 `-(unsigned int)value` é a forma segura de obter o valor absoluto. Escrever `-value` rebentaria com o menor `int` possível, que não tem simétrico.
- 4 O resto é o padrão de sempre: contar algarismos, pedir memória, preencher de trás para a frente.

O CÓDIGO

ft_itoa_base.c

```
1  #include <stdlib.h>
2
3  char  *ft_itoa_base(int value, int base)
4  {
5      char      *s;
6      char      *digits;
7      unsigned int  n;
8      int         len;
9      int         neg;
10
11     digits = "0123456789ABCDEF";
12     neg = (base == 10 && value < 0);
13     n = (unsigned int)value;
14     if (neg)
15         n = -(unsigned int)value;
16     len = 1;
17     while (n >= (unsigned int)base)
18     {
19         n = n / (unsigned int)base;
20         len++;
21     }
22     len = len + neg;
23     s = malloc(sizeof(char) * (len + 1));
24     if (!s)
25         return (0);
26     s[len] = '\0';
27     n = (unsigned int)value;
28     if (neg)
29         n = -(unsigned int)value;
30     while (len-- > neg)
31     {
32         s[len] = digits[n % (unsigned int)base];
33         n = n / (unsigned int)base;
34     }
35     if (neg)
36         s[0] = '-';
37     return (s);
38 }
```

LINHA A LINHA

- | | |
|-------|--|
| 11 | A tabela dos dígitos, com maiúsculas como o enunciado pede. |
| 12 | O sinal só existe em base 10. Esta linha é a regra do enunciado escrita tal e qual. |
| 13-15 | O número passa a <code>unsigned int</code> . Em base 10 negativa usa-se o valor absoluto; nas outras usa-se o valor tal como está, reinterpretado como positivo. |
| 17-21 | Conta quantos dígitos tem nesta base. É o mesmo ciclo do <code>ft_itoa</code> com <code>base</code> em vez de 10. |
| 22 | Mais um caractere se houver sinal. |
| 23 | Pede o espaço, com o <code>+ 1</code> do <code>'\0'</code> . |
| 27-29 | Repõe o valor, porque o ciclo da contagem o destruiu. |
| 30 | <code>len-- > neg</code> compara e depois recua; pára em <code>neg</code> , deixando livre a posição do sinal. |
| 32 | O resto da divisão pela base dá um número entre o e <code>base - 1</code> , que é exactamente a posição certa na tabela. |

O QUE O ENUNCIADO PEDE

Escrever uma função que recebe um endereço e um tamanho e mostra a memória como no exemplo do enunciado: cada linha tem 16 bytes em hexadecimal, agrupados dois a dois, seguidos dos mesmos 16 bytes como texto.

Os bytes que não sejam caracteres visíveis aparecem como `.`. A última linha, se tiver menos de 16 bytes, leva espaços no lugar dos que faltam para o texto ficar alinhado.

A declaração é `void print_memory(const void *addr, size_t size);`

Funções permitidas: `write`.

```
0000 0000 1700 0000 9600 0000 ff00 0000 .....  
0c00 0000 1000 0000 1500 0000 2a00 0000 .....*..  
0000 0000 0000 0000 .....  
.....
```

COMO PENSAR

- 1 O exercício não tem algoritmo nenhum: é tudo formato. Ganha-se ou perde-se no alinhamento.
- 2 Três ciclos encaixados: um por linha de 16 bytes, um para a parte hexadecimal, e um para a parte de texto.
- 3 O `const void *` é um endereço sem tipo. Para o poder ler byte a byte, copia-se para um `unsigned char *` — assim cada posição é exactamente um byte.
- 4 O detalhe que faz o alinhamento da última linha: quando já não há bytes, escrevem-se **dois espaços** no lugar dos dois dígitos hexadecimais. Assim a coluna do texto começa sempre na mesma posição.

O CÓDIGO

print_memory.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  void    put_hex_byte(unsigned char b)
9  {
10     char    *digits;
11
12     digits = "0123456789abcdef";
13     ft_putchar(digits[b / 16]);
14     ft_putchar(digits[b % 16]);
15 }
16
17 void    print_memory(const void *addr, size_t size)
18 {
19     const unsigned char *p;
20     size_t                i;
21     size_t                j;
22
23     p = addr;
24     i = 0;
25     while (i < size)
26     {
27         j = 0;
28         while (j < 16)
29         {
30             if (i + j < size)
31                 put_hex_byte(p[i + j]);
32             else
33                 write(1, " ", 2);
34             if (j % 2 == 1)
35                 ft_putchar(' ');
36             j++;
37         }
38         j = 0;
39         while (j < 16 && i + j < size)
40         {
41             if (p[i + j] >= 32 && p[i + j] <= 126)
42                 ft_putchar(p[i + j]);
43             else
44                 ft_putchar('.');
45             j++;
46         }
47         ft_putchar('\n');
48         i = i + 16;
49     }
50 }
```

LINHA A LINHA

8-15	Escreve um byte como dois dígitos hexadecimais. <code>b / 16</code> é o dígito da esquerda e <code>b % 16</code> o da direita.
19	O ponteiro com tipo, para se poder ler byte a byte.
25	Um ciclo por cada linha de 16 bytes.
28	O ciclo da parte hexadecimal: corre sempre 16 vezes, haja bytes ou não.
30-31	Se ainda há byte, escreve-o.
32-33	Senão escreve dois espaços, para manter o alinhamento.
34-35	Um espaço a cada dois bytes, que é o agrupamento do enunciado.
39	O ciclo do texto: este pára quando os bytes acabam, não preenche nada.
41-42	Caracteres visíveis vão de 32 a 126.
44	Tudo o resto sai como ponto.
48	Salta 16 bytes para a linha seguinte.

N5 options

Juntar opções da linha de comando num inteiro e mostrá-lo em binário.

O QUE O ENUNCIADO PEDE

O programa recebe um número indefinido de argumentos que são opções, e escreve uma representação dessas opções em grupos de bytes, seguida de um newline.

Uma opção é um argumento que começa por `-` e tem um ou mais caracteres de `a` a `z`. Todas as opções são guardadas num só `int`, em que cada opção é um bit: o `a` é o bit 0, o `b` o bit 1, e assim por diante até ao `z`.

Sem argumentos, ou com a opção `-h` activa, mostra-se uma linha de uso. Uma opção inválida escreve `Invalid Option` e um newline.

Funções permitidas: `write`.

```
$> ./options -abc -ijk
00000000 00000000 00000111 00000111
$> ./options -z
00000010 00000000 00000000 00000000
$> ./options -%
Invalid Option
```

COMO PENSAR

- 1 A ideia central: um `int` tem 32 bits e o alfabeto tem 26 letras, por isso cabe uma letra por bit.
- 2 Ligar o bit da letra `c` escreve-se `flags = flags | (1 << (c - 'a'))`. O `c - 'a'` dá a posição da letra (o para o `a`), o `1 <<` põe um 1 nessa posição, e o `|` acende-o sem apagar os outros.
- 3 Escrever em binário é o `print_bits` do nível 2 esticado para 32 bits, com um espaço a cada oito.
- 4 O `-h` tem prioridade: assim que aparece, mostra-se o uso e acaba, mesmo que já tenham sido lidas outras opções.

O CÓDIGO

`options.c`

```

1  #include <unistd.h>
2  #include "options.h"
3
4  void    ft_putstr(char *s)
5  {
6      int i;
7
8      i = 0;
9      while (s[i])
10         i++;
11     write(1, s, i);
12 }
13
14 void    put_flags(unsigned int n)
15 {
16     int    i;
17     char    c;
18
19     i = 31;
20     while (i >= 0)
21     {
22         c = ((n >> i) & 1) + '0';
23         write(1, &c, 1);
24         if (i % 8 == 0 && i != 0)
25             write(1, " ", 1);
26         i--;
27     }
28     write(1, "\n", 1);
29 }
30
31 void    usage(void)
32 {
33     ft_putstr("options: abcdefghijklmnopqrstuvwxyz\n");
34 }
35
36 int     main(int argc, char **argv)
37 {
38     unsigned int    flags;
39     int             i;
40     int             j;
41
42     flags = 0;
43     if (argc == 1)
44     {
45         usage();
46         return (0);
47     }
48     i = 1;
49     while (i < argc)
50     {
51         j = 1;
52         if (argv[i][0] != '-' || argv[i][1] == '\0')
53         {
54             ft_putstr("Invalid Option\n");
55             return (0);
56         }
57         while (argv[i][j])
58         {
59             if (argv[i][j] < 'a' || argv[i][j] > 'z')
60             {
61                 ft_putstr("Invalid Option\n");
62                 return (0);
63             }
64             if (argv[i][j] == 'h')
65             {
66                 usage();
67                 return (0);

```

options.h

LINHA A LINHA

N5 brainfuck

Interpretador da linguagem Brainfuck.

O QUE O ENUNCIADO PEDE

Escrever um interpretador de Brainfuck. O código-fonte vem no primeiro argumento e é sempre válido.

A linguagem tem uma fita de 2048 bytes a zero e um ponteiro para o primeiro. Os operadores são: `>` avança o ponteiro; `<` recua; `+` incrementa o byte apontado; `-` decrementa; `.` escreve o byte apontado; `[` salta para o `]` correspondente se o byte for zero; `]` volta ao `[` correspondente se o byte não for zero. Qualquer outro caractere é comentário.

Funções permitidas: `write`, `malloc`, `free`.

COMO PENSAR

- 1 Seis dos oito operadores são uma linha cada: uma cadeia de `else if` resolve-os.
- 2 A dificuldade está só nos parênteses rectos, porque é preciso encontrar o **correspondente**, e pode haver ciclos dentro de ciclos.
- 3 A solução é contar a profundidade: ao procurar para a frente, começa-se em 1, cada `[` novo soma e cada `]` subtrai; quando chegar a zero, é o correspondente. Ao procurar para trás faz-se o mesmo ao contrário.

④ Duas variáveis a não confundir: o `pc` é onde se está no **código**, o `p` é onde se está na **fita**.

O CÓDIGO

brainfuck.c

```

1  #include <unistd.h>
2  #include <stdlib.h>
3  #include "brainfuck.h"
4
5  void  jump_fwd(char *code, int *pc)
6  {
7      int depth;
8
9      depth = 1;
10     while (depth > 0 && code[*pc])
11     {
12         (*pc)++;
13         if (code[*pc] == '[')
14             depth++;
15         else if (code[*pc] == ']')
16             depth--;
17     }
18 }
19
20 void  jump_back(char *code, int *pc)
21 {
22     int depth;
23
24     depth = 1;
25     while (depth > 0 && *pc > 0)
26     {
27         (*pc)--;
28         if (code[*pc] == ']')
29             depth++;
30         else if (code[*pc] == '[')
31             depth--;
32     }
33 }
34
35 int  main(int argc, char **argv)
36 {
37     char    *mem;
38     char    *code;
39     int     p;
40     int     pc;
41
42     if (argc != 2)
43     {
44         write(1, "\n", 1);
45         return (0);
46     }
47     mem = malloc(2048);
48     if (!mem)
49         return (1);
50     p = 0;
51     while (p < 2048)
52         mem[p++] = 0;
53     p = 0;
54     pc = 0;
55     code = argv[1];
56     while (code[pc])
57     {
58         if (code[pc] == '>')
59             p++;
60         else if (code[pc] == '<')
61             p--;
62         else if (code[pc] == '+')
63             mem[p]++;
64         else if (code[pc] == '-')
65             mem[p]--;
66         else if (code[pc] == '.')
67             write(1, &mem[p], 1);

```

```

68     else if (code[pc] == '[' && mem[p] == 0)
69         jump_fwd(code, &pc);
70     else if (code[pc] == ']' && mem[p] != 0)
71         jump_back(code, &pc);
72     pc++;
73 }
74 free(mem);
75 return (0);
76 }

```

brainfuck.h

```

1  #ifndef BRAINFUCK_H
2  # define BRAINFUCK_H
3
4  void  jump_fwd(char *code, int *pc);
5  void  jump_back(char *code, int *pc);
6
7  #endif

```

LINHA A LINHA

- 5-18 Procura o `]` correspondente, para a frente.
- 9 Começa em 1 porque já se está em cima de um `[`.
- 13-16 Cada `[` aumenta a profundidade, cada `]` diminui. Quando chega a zero, parou no sítio certo.
- 20-33 O mesmo ao contrário, para trás.
- 47-52 A fita. O `malloc` não põe zeros, por isso o ciclo da linha 51 tem de o fazer à mão.
- 58-65 Os quatro operadores de movimento e de valor, um por linha.
- 66-67 O `.` escreve o byte apontado. `&mem[p]` é o endereço dele.
- 68-69 O `[` só salta se o byte for zero; senão não faz nada e o ciclo continua normalmente.
- 70-71 O `]` só volta atrás se o byte não for zero.
- 72 Este `pc++` corre sempre, inclusive depois de um salto. É o que faz o programa continuar a partir do parêntese correspondente.

Qualquer caractere que não seja um dos oito operadores não bate certo com nenhum `else if` e é simplesmente ignorado — os comentários ficam tratados sem uma única linha de código.

N5 rpn_calc

Avaliar uma expressão em notação polaca inversa.

O QUE O ENUNCIADO PEDE

Receber uma expressão escrita em notação polaca inversa no primeiro argumento, calculá-la e mostrar o resultado seguido de um newline.

Na notação polaca inversa o operador vem **depois** dos seus dois operandos: `3 4 +` vale 7. Cada operador consome os dois valores anteriores e o resultado fica no lugar deles. Os operandos e operadores estão separados por pelo menos um espaço.

Há que implementar `+`, `-`, `*`, `/` e `%`. Se a expressão não for válida, ou se não houver exactamente um argumento, mostrar `Error` seguido de um newline.

Funções permitidas: `atoi`, `printf`, `write`, `malloc`, `free`.

```
$> ./rpn_calc "1 2 * 3 * 4 +" | cat -e
10$
$> ./rpn_calc "1 2 3 4 +" | cat -e
Error$
```

COMO PENSAR

- ❶ Outra vez uma pilha, como no `brackets`, mas agora de números.
- ❷ A regra é simples: se o pedaço lido é um número, empilha-se; se é um operador, tiram-se os dois de cima, fazem a conta e empilha-se o resultado.
- ❸ No fim tem de sobrar **exactamente um** valor. É isso que faz o segundo exemplo dar erro: sobram quatro números e nenhum operador que os consuma.
- ❹ Para partir a string em pedaços sem funções de biblioteca: saltar espaços, medir até ao próximo espaço, e avançar esse tanto.
- ❺ Decidir se um pedaço é número ou operador antes de o usar é o que apanha todas as entradas inválidas com uma verificação só.

O CÓDIGO

rpn_calc.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "rpn_calc.h"
4
5  int tok_len(char *s, int i)
6  {
7      int n;
8
9      n = 0;
10     while (s[i + n] && s[i + n] != ' ')
11         n++;
12     return (n);
13 }
14
15 int is_number(char *s, int i)
16 {
17     int n;
18     int k;
19
20     n = tok_len(s, i);
21     if (n == 0)
22         return (0);
23     k = 0;
24     if (s[i] == '-' && n > 1)
25         k = 1;
26     while (k < n)
27     {
28         if (s[i + k] < '0' || s[i + k] > '9')
29             return (0);
30         k++;
31     }
32     return (1);
33 }
34
35 int is_operator(char *s, int i)
36 {
37     if (tok_len(s, i) != 1)
38         return (0);
39     return (s[i] == '+' || s[i] == '-' || s[i] == '*'
40             || s[i] == '/' || s[i] == '%');
41 }
42
43 int apply(int a, int b, char op, int *err)
44 {
45     if (op == '+')
46         return (a + b);
47     if (op == '-')
48         return (a - b);
49     if (op == '*')
50         return (a * b);
51     if (b == 0)
52     {
53         *err = 1;
54         return (0);
55     }
56     if (op == '/')
57         return (a / b);
58     return (a % b);
59 }
60
61 int main(int argc, char **argv)
62 {
63     int stack[512];
64     int top;
65     int i;
66     int err;
67

```

```

68     err = 0;
69     top = 0;
70     i = 0;
71     if (argc != 2)
72         err = 1;
73     while (!err && argv[1][i])
74     {
75         while (argv[1][i] == ' ')
76             i++;
77         if (!argv[1][i])
78             break ;
79         if (is_number(argv[1], i) && top < 512)
80             stack[top++] = atoi(&argv[1][i]);
81         else if (is_operator(argv[1], i) && top >= 2)
82         {
83             stack[top - 2] = apply(stack[top - 2], stack[top - 1],
84                                   argv[1][i], &err);
85             top--;
86         }
87         else
88             err = 1;
89         i = i + tok_len(argv[1], i);
90     }
91     if (err || top != 1)
92         printf("Error\n");
93     else
94         printf("%d\n", stack[0]);
95     return (0);
96 }

```

rpn_calc.h

```

1  #ifndef RPN_CALC_H
2  # define RPN_CALC_H
3
4  int  tok_len(char *s, int i);
5  int  is_number(char *s, int i);
6  int  is_operator(char *s, int i);
7  int  apply(int a, int b, char op, int *err);
8
9  #endif

```

LINHA A LINHA

- | | |
|-------|---|
| 5-13 | Mede o pedaço que começa na posição <code>i</code> : conta até ao próximo espaço ou ao fim. |
| 15-33 | É um número? Aceita um <code>-</code> à frente, mas só se houver algarismos a seguir — assim o <code>-</code> sozinho não passa por número. |
| 35-41 | É um operador? Tem de ter exactamente um caractere e ser um dos cinco. |
| 43-59 | Faz a conta. A divisão e o resto por zero marcam erro em vez de rebentar o programa. |
| 63-64 | A pilha de números e o índice do topo. |
| 73 | O ciclo pára assim que houver erro, sem ler o resto. |
| 75-78 | Salta os espaços. Se chegou ao fim, acaba. |
| 79-80 | Número: empilha. O <code>atoi</code> lê a partir da posição actual e pára sozinho no espaço. |
| 81-86 | Operador: aplica aos dois de cima. O resultado fica na posição do primeiro dos dois e o topo desce um. |
| 87-88 | Não é número nem operador, ou não havia dois valores para o operador: erro. |
| 89 | Avança exactamente o comprimento do pedaço lido. |
| 91-92 | Erro se houve algum problema, ou se não sobrou exactamente um valor. |

O enunciado não diz o que fazer numa divisão por zero. Aqui optou-se por `Error`, que é o mais seguro: qualquer outra escolha faria o programa rebentar e um programa que rebenta falha sempre.

A folha de cola

Uma página com tudo o que é preciso escrever de cor. Imprime só esta.

O ARRANQUE — ESCREVER ISTO PRIMEIRO, SEMPRE

```
#include <unistd.h>

void ft_putchar(char c)
{
    write(1, &c, 1);
}

int main(int argc, char **argv)
{
    int i;

    if (argc == 2)
    {
        i = 0;
        while (argv[1][i])
        {
            /* o exercicio e so isto */
            i++;
        }
        ft_putchar('\n');
        return (0);
    }
}
```

ESCREVER UM NÚMERO

```
void ft_putnbr(int n)
{
    if (n < 0)
    {
        ft_putchar('-');
        n = -n;
    }
    if (n >= 10)
        ft_putnbr(n / 10);
    ft_putchar(n % 10 + '0');
}
```

SEPARADOR DE PALAVRAS

```
int is_space(char c)
{
    return (c == ' ' || c == '\t');
}
```

SALTAR SEPARADORES, DEPOIS A PALAVRA

```
while (is_space(s[i]))
    i++;
while (s[i] && !is_space(s[i]))
{
    ft_putchar(s[i]);
    i++;
}
```

TEXTO → NÚMERO

```
n = 0;
i = 0;
while (s[i] >= '0' && s[i] <= '9')
{
    n = n * 10 + (s[i] - '0');
    i++;
}
```

ARITMÉTICA DE LETRAS

é minúscula?	<code>c>='a' && c<='z'</code>
é maiúscula?	<code>c>='A' && c<='Z'</code>
é algarismo?	<code>c>='0' && c<='9'</code>
minúscula → MAIÚSCULA	<code>c - 32</code>
MAIÚSCULA → minúscula	<code>c + 32</code>
posição no alfabeto	<code>c - 'a'</code>
letra n.º i	<code>'a' + i</code>
algarismo → número	<code>c - '0'</code>
número → algarismo	<code>n + '0'</code>
rodar k letras	<code>'a'+(c-'a'+k)%26</code>
espelhar a letra	<code>'z' - (c - 'a')</code>

ARGC CONTA O NOME DO PROGRAMA

"1 argumento"	<code>argc == 2</code>
"2 argumentos"	<code>argc == 3</code>
"3 argumentos"	<code>argc == 4</code>
primeiro / último	<code>argv[1] / argv[argc-1]</code>
recebido e não usado	<code>(void)argv;</code>

BITS

ver o bit i	<code>(o >> i) & 1</code>
trocar metades	<code>(o>>4) (o<<4)</code>
é potência de 2	<code>n && !(n & (n-1))</code>

MALLOC — TRÊS PASSOS, POR ESTA ORDEM

1. contar `len = ... (+1 se for string)`
2. pedir `p = malloc(sizeof(t) * len);`
 `if (!p) return (0);`
3. encher e por o `'\0'` a mao

AS TRÊS RECEITAS DE MATEMÁTICA

```
/* mdc de a e b */
while (b != 0)
{
    t = a % b;
    a = b;
    b = t;
}

/* mmc = a / mdc * b (dividir primeiro!) */

/* e primo? */
i = 2;
while (i * i <= n)
{
    if (n % i == 0)
        return (0);
    i++;
}
```

Antes de submeter: zero avisos com `cc -Wall -Wextra -Werror` · todos os exemplos com `| cat -e` · sem argumentos e com argumentos a mais · apagar o `main` de teste se o exercício for uma função · na pasta, só os `Expected files`.

Índice alfabético

Os 70 exercícios por ordem alfabética, com o nível de cada um. Também aqui o rato por cima de um nome mostra o enunciado.

3	add_prime_sum	0	ft_print_numbers	3	pgcd
0	aff_a	1	ft_putstr	2	print_bits
0	aff_first_param	3	ft_range	3	print_hex
0	aff_last_param	3	ft_rrange	5	print_memory
0	aff_z	4	ft_split	1	repeat_alpha
2	alpha_mirror	2	ft_strcmp	1	rev_print
5	brackets	1	ft_strcpy	4	rev_wstr
5	brainfuck	2	ft_strdup	2	reverse_bits
4	check_mate	1	ft_strlen	4	rostring
2	do_op	2	ft_strrev	1	rot_13
3	epur_str	1	ft_swap	1	rotone
3	expand_str	0	hello	5	rpn_calc
1	first_word	3	hiddenp	3	rstr_capitalizer
1	fizzbuzz	2	inter	1	search_and_replace
4	flood_fill	2	is_power_of_2	4	sort_int_tab
4	fprime	2	last_word	4	sort_list
2	ft_atoi	3	lcm	3	str_capitalizer
3	ft_atoi_base	0	maff_alpha	2	swap_bits
0	ft_countdown	0	maff_revalpha	3	tab_mult
4	ft_itoa	2	max	1	ulstr
5	ft_itoa_base	0	only_a	2	union
4	ft_list_foreach	0	only_z	2	wdmatch
4	ft_list_remove_if	5	options		
3	ft_list_size	3	paramsum		

Os 70 ficheiros `.c` e `.h` deste documento compilam com `gcc -Wall -Wextra -Werror` sem um único aviso e passam 149 comparações de output tiradas dos exemplos dos enunciados, mais os testes de retorno das funções. O código impresso é injectado a partir desses mesmos ficheiros testados.

A pool pode variar ligeiramente entre campus e entre anos.